INSTITUTO TECNOLÓGICO SUPERIOR TECSUP

SOFTWARE DESIGN AND DEVELOPMENT



COURSE: CLOUD SOLUTIONS DEVELOPMENT

PROJECT:

LeGrandHotel – Web-based hotel management system

LeGrandHotel Week11

Features, tests and mobile design

AREQUIPA, 2026

Lecturer:

VÍCTOR CALVO

Presented by:

- Huacca Ccaso, Diego Armando
- Marcani Leyva, Joseph Fredy
- Pinto Garate, Leo Mathiu
- Quispe Encarnación, Rusell

Arequipa – Peru

2026

Table of contents

1. Project description 3
1.1 Objectives 3
1.2 Modules 3
1.3 Technologies 3
1.4 How the parts connect 4
1.5 Database diagram 5
2. Defined endpoints 8
2.1 Single response format 8
3. Separate registration and login 9
3.1 Code excerpts 10
3.2 Registration screens 11
4. What each role can do 11
4.1 Receptionist 12
4.2 Storekeeper 21
4.3 Administrator 24
4.4 Developer 42
4.5 Guest 47
5. Error screens 49
6. Incident log 51
7. Performance and stress tests with k6 52
7.1 Smoke test 53
7.2 Login attempt limit 54
7.3 Load steps (5, 10, 25 and 50 users) 55
7.4 k6 HTML report (50-user step) 57
7.5 Test conclusions 59
8. Android and iOS app design 59
8.1 Technology 62
8.2 Security and API changes 62
8.3 Build plan 62
9. Conclusions 63

1. Project description

LeGrandHotel is a web system to manage a hotel. It brings together the staff's daily work (reservations, check-in and check-out, rooms, guests, parking and inventory) and online booking for guests, who choose a room type, see what it includes and the total, and the hotel assigns them a free room.

The system has two parts: a REST API that stores the data and enforces every business rule, and a web page (single-page application) that uses it. The same API will also serve the Android and iOS mobile app designed in section 8.

1.1 Objectives

- Prevent operational errors: a room cannot be booked twice for the same dates, two guests cannot be in the same room and inventory cannot go negative.
- Each person sees only what their role allows, and the API enforces it even if someone types an address by hand.
- Leave a trail: every change is recorded in the audit log with who made it and when, and every error has an incident code to find it in the log.
- Measure performance with load tests before publishing the system.

1.2 Modules

Module	What it covers
Reservations and stays	Reservations with the total calculated on the server, check-in (creates the stay and occupies the room) and check-out (the room goes to cleaning)
Rooms and room types	Rooms by floor and status; types with price per night, capacity and amenities
Guests	Guest records with document, phone and nationality; linked to the web account if the guest has one
Parking	Parking spots, vehicle entry and exit, with or without a related stay
Inventory	Supplies with stock and minimum; stock in and out with a reason
Staff and accounts	Staff records, account registration and role changes
Reports and audit	Occupancy, income for the last 6 months and who changed what
Guest portal	Google sign-in, guest details, online booking and their reservations
Developer view	API health, latencies, requests, errors and routes, without logging into the server

1.3 Technologies

Part	Technology	What it is used for
API (backend)	PHP 8.2 and Laravel 12	REST routes, validation, business rules and data access
Authentication	Laravel Sanctum	Bearer tokens that expire (60 minutes) for the web and the mobile app
Google sign-in	Laravel Socialite (OAuth 2.0)	Guest accounts without a password
Database	MySQL 8.4	Hotel data with foreign keys, indexes and transactions
Web page (frontend)	React 18, TypeScript and Vite	Fast, typed single-page interface

Part	Technology	What it is used for
Data on the web	TanStack Query and Axios	Query cache, pagination and a single handler for 401/403/429/5xx errors
Navigation	React Router	Role-protected routes and error screens
API tests	Pest (PHPUnit)	121 automated tests of rules, permissions and errors
Web tests	Vitest and Testing Library	25 tests of routes, forms and permissions
Code quality	PHPStan level 5, Laravel Pint, ESLint and strict TypeScript	Catch errors before running and keep one style
Load tests	k6 (Grafana)	Measure response times and errors with many users at once
Local environment	Docker Compose	The same MySQL database for the whole team
Mobile app (design)	React Native with Expo and TypeScript	One codebase for Android and iOS
Version control	Git and GitHub	History and teamwork

Note: the server where the system will be published has not been defined yet. Everything in this report was run in the team's local environment.

1.4 How the parts connect

The web page and the mobile app do not store data: they request everything from the API in JSON with the session token. The API validates every request (FormRequest), checks the role (role: middleware) and the per-record rules (policies), and always answers in the same format, whether success or error. The MySQL database has 16 system tables related by 13 foreign keys; the diagram in section 1.5 was generated by reading the real database schema.

1.5 Database diagram

The diagram is split into three parts so it is easy to read. When a table relates to one in another part, that table appears in a dashed box with only its id. Each line shows the foreign key column and what MySQL does when the parent record is deleted (CASCADE, SET NULL or RESTRICT). Table and column names are the real ones in the database.

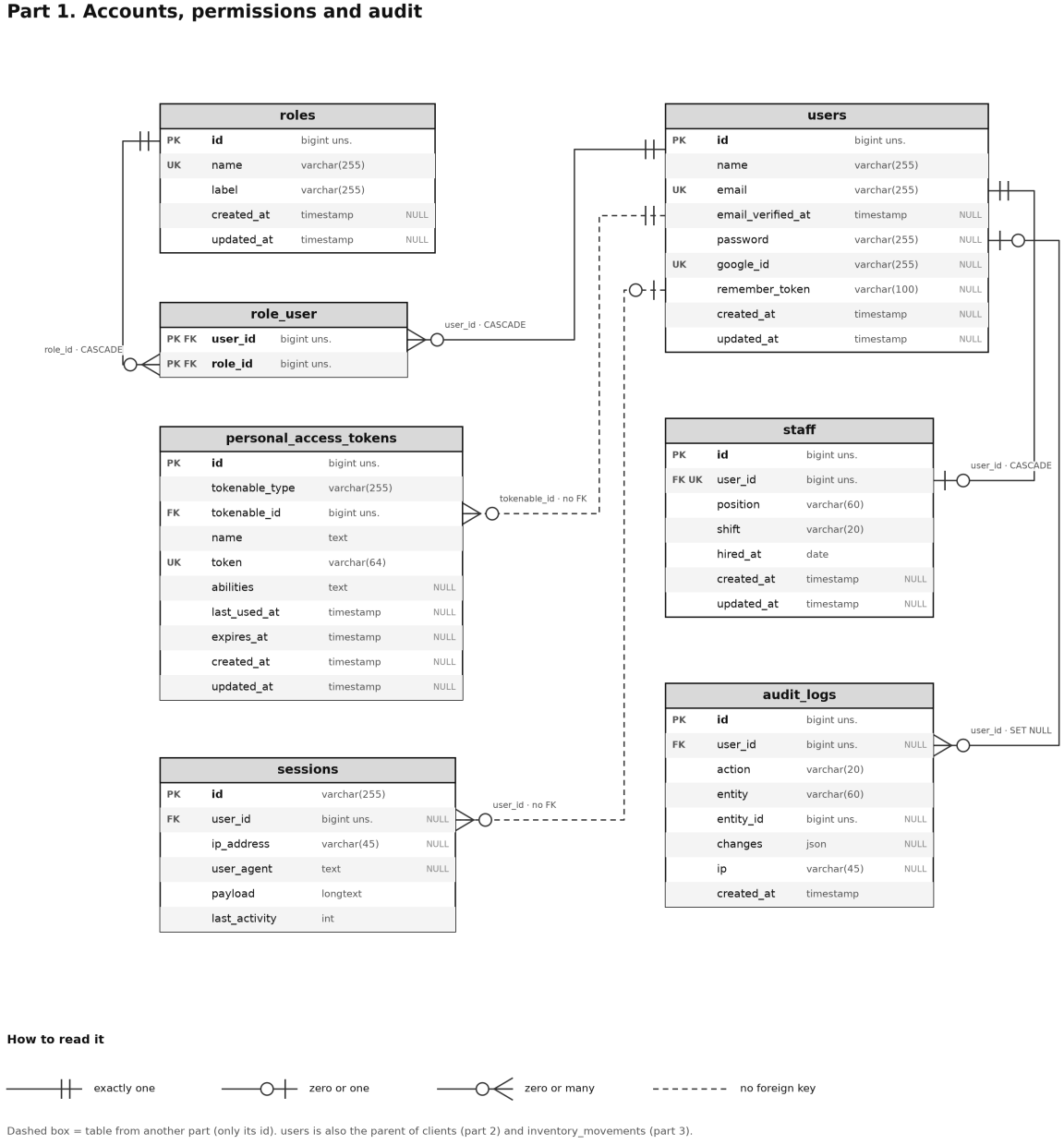


Figure 1. Database diagram, part 1: accounts, permissions and audit.

Part 2. Guests, reservations and rooms

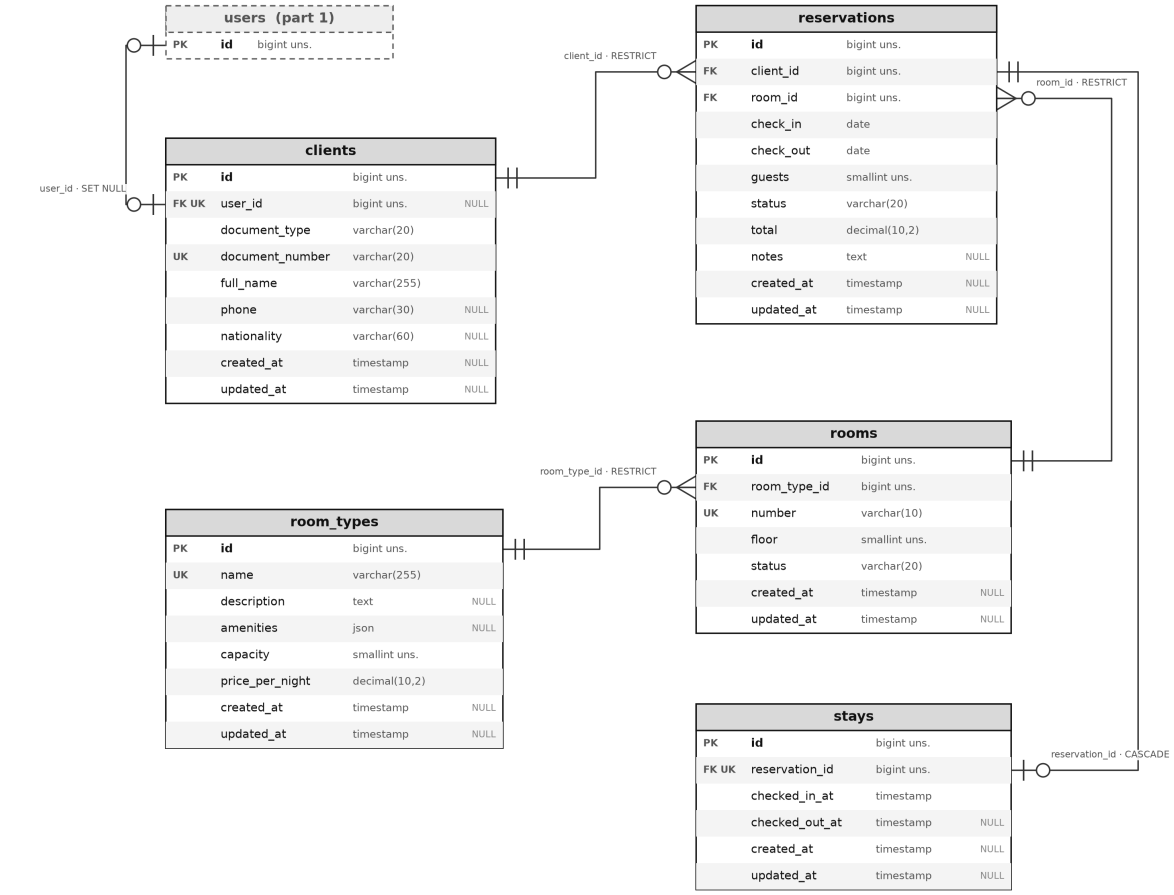


Figure 2. Database diagram, part 2: guests, reservations and rooms.

Part 3. Parking and inventory

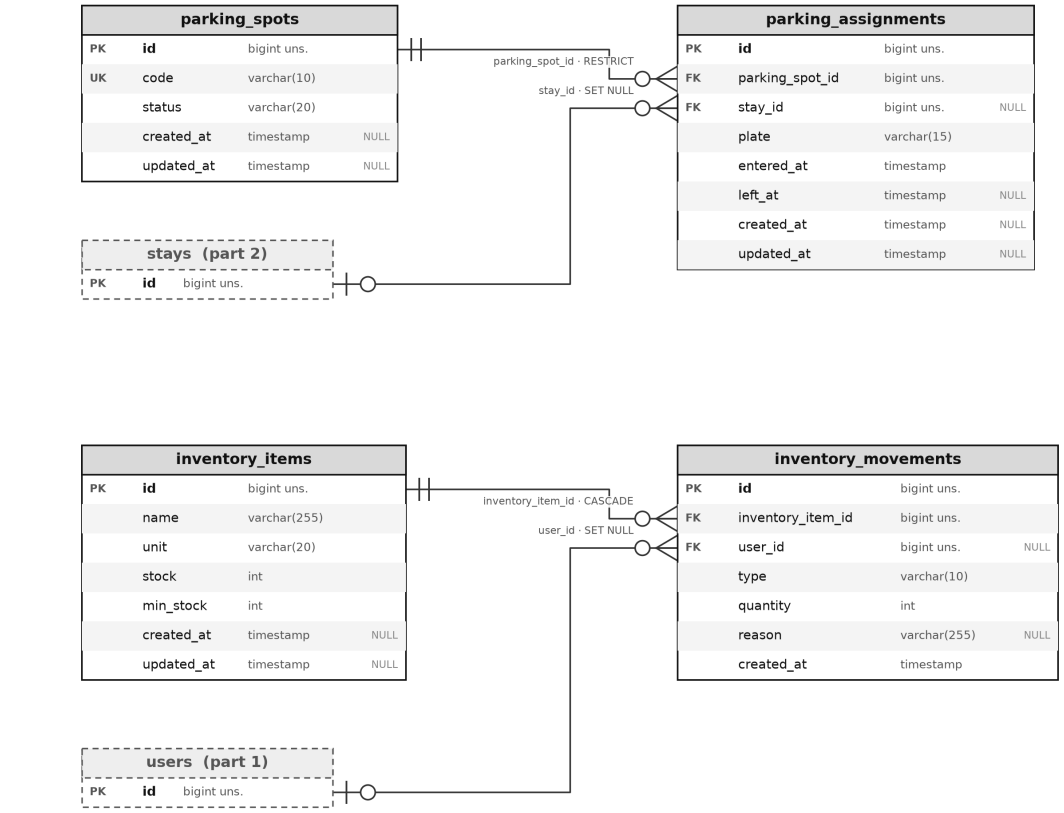


Figure 3. Database diagram, part 3: parking and inventory.

2. Defined endpoints

All routes live under /api/v1 and answer JSON. There are 77 routes (counted with php artisan route:list). In the table, “CRUD” means list (GET), create (POST), show (GET /{id}), update (PUT/PATCH) and delete (DELETE).

Group	Method and route	Who can use it
Health	`GET /health`	Public
Authentication	`POST /auth/login`	Public (5 attempts per minute)
	`GET /auth/google/redirect · /auth/google/callback`	Public (guests)
	`GET /auth/me · POST /auth/logout · POST /auth/refresh`	Any session
	`POST /auth/register`	Administrator and Developer (3 per minute)
Email	`GET /email/verify/{id}/{hash} · POST /email/verification-notification`	Signed link · session
Guest portal	`PUT /portal/profile`	Guest
	`GET /portal/room-types`	Guest
	`GET · POST /portal/reservations`	Guest
	`POST /portal/reservations/{id}/cancel`	Guest (own reservations only)
Dashboard	`GET /dashboard`	Administrator, Receptionist, Storekeeper, Developer
Room types	`GET /room-types · /room-types/{id}`	Administrator and Receptionist
	`POST · PUT · DELETE /room-types`	Administrator
Reservations	`CRUD /reservations · POST /reservations/{id}/check-in`	Administrator and Receptionist
Stays	`GET /stays · /stays/{id} · POST /stays/{id}/check-out`	Administrator and Receptionist
Rooms	`CRUD /rooms · GET /rooms/available · PATCH /rooms/{id}/status`	Administrator and Receptionist
Guests	`CRUD /clients`	Administrator and Receptionist
Parking	`CRUD /parking-spots · CRUD /parking-assignments`	Administrator and Receptionist
Inventory	`CRUD /inventory-items · GET · POST /inventory-items/{id}/movements`	Administrator and Storekeeper
Staff	`CRUD /staff`	Administrator
Users	`CRUD /users · PUT /users/{id}/roles`	Administrator
Reports	`GET /reports/occupancy · /reports/income`	Administrator
Audit	`GET /audit-logs`	Administrator
Developer	`GET /dev/overview · /dev/requests · /dev/errors · /dev/routes`	Developer and Administrator

2.1 Single response format

Every response uses App\Support\ApiResponse. A success carries success and data (plus meta in paginated lists, with at most 50 per page); an error carries a stable code that the web uses to

decide what to show, never HTML or server traces. Messages are in Spanish because the system is for a Peruvian hotel.

Error response (real example: double booking)

```
{
  "success": false,
  "error": {
    "code": "ROOM_ALREADY_BOOKED",
    "message": "La habitación ya está reservada en esas fechas."
  }
}
```

HTTP code	When the API returns it	What the web shows
401	No session or expired token	“Your session ended” screen and back to login
403	The role is not allowed	“You do not have access to this section” screen
404	The record or route does not exist	“We could not find this page” screen
409	Business rule: double booking, occupied room, insufficient stock, single Developer…	Message in the form
422	Invalid data (FormRequest)	Error next to each field
429	Too many attempts (Retry-After)	Screen with a countdown
500	Unexpected error, with an incident code	500 screen with the code for support
503	Service unavailable or offline	Screen that retries by itself

3. Separate registration and login

There are two kinds of account with separate paths, both on screen and in the API. The rules are validated on the server: even if someone modifies the page, the API rejects what is not allowed.

	Hotel staff	Guest
Roles	Administrator, Receptionist, Storekeeper, Developer	Client (guest)
How they sign in	Email and password (POST /auth/login)	Only with Google (OAuth); no password
Who creates the account	The Administrator or the Developer, from “Register staff”. There is no public sign-up	Created automatically the first time they sign in with Google
Extra data	Staff record: position, shift and start date	DNI (8 digits) and phone (9 digits), only once
Protection	5 login attempts per minute; bcrypt-hashed password; at least 8 characters with letters and numbers	Google verifies the email; a DNI cannot be linked to two accounts
Where they land	Their role’s dashboard	“Complete registration”, then “Book”

There is only one Developer. Neither the Administrator nor the Developer can create another Developer account, give that role to someone else, remove it from the existing one or delete it. The API answers 409 DEVELOPER_UNIQUE or DEVELOPER_LOCKED if someone tries.

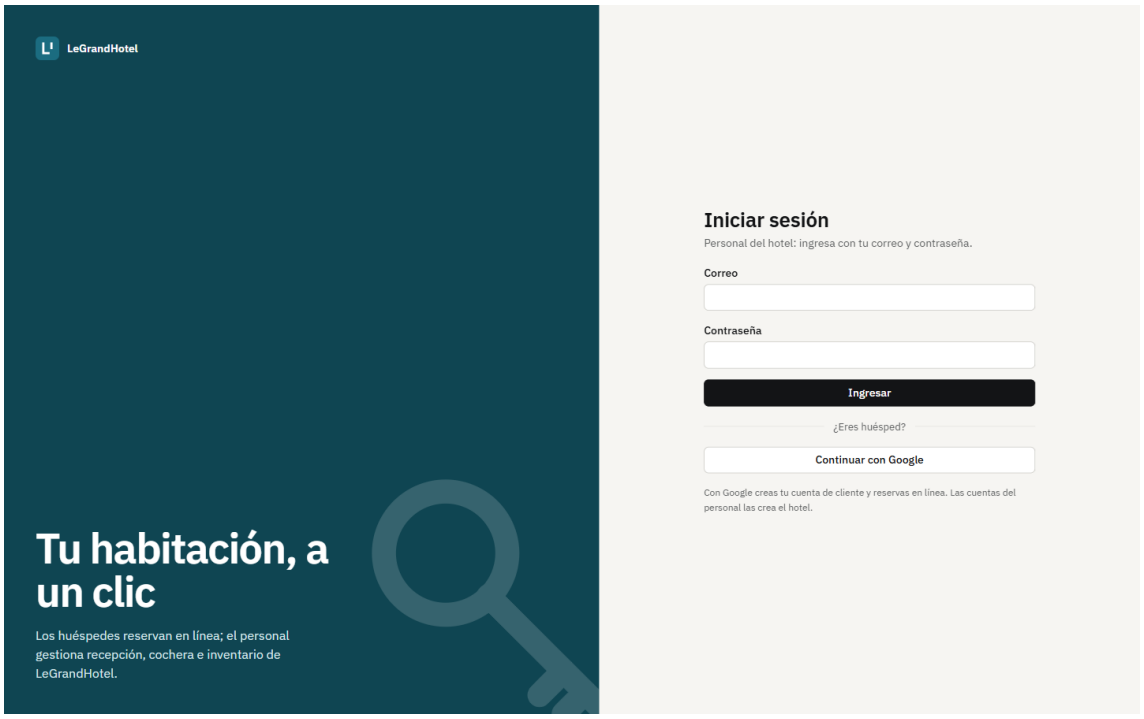


Figure 4. Login: staff sign in with email and password; guests with "Continuar con Google". The interface is in Spanish.

3.1 Code excerpts

Password login only lets staff in; a guest who tries it gets 403:

api/app/Http/Controllers/Api/AuthController.php

```
if (! $user || ! $user->password || ! Hash::check($request->string('password'), $user-
>password)) {
    throw new ApiException('INVALID_CREDENTIALS', 'Correo o contraseña incorrectos.',
401);
}
// Only staff sign in with a password; guests sign in with Google (they have no
password).
if (! $user->hasRole(...Role::INTERNAL)) {
    throw new ApiException('ACCOUNT_WITHOUT_ACCESS', 'Esta cuenta no tiene acceso al
sistema del hotel.', 403);
}
```

Staff registration only accepts the allowed roles, and the route is protected by role and by an attempt limit:

api/routes/api.php and api/app/Http/Requests/Auth/RegisterRequest.php

```
Route::post('register', [AuthController::class, 'register'])
    ->middleware(['role:admin,developer', 'throttle:register']);

'password' => ['required', 'confirmed', Password::min(8)->letters()->numbers()],
'role'     => ['required', Rule::in(Role::ASSIGNABLE)],   // Front desk, Storeroom or
Administration
```

Google is only for guests: if the email already belongs to a staff member, they are sent back to the login with a warning:

api/app/Http/Controllers/Api/GoogleAuthController.php

```
$user = User::where('google_id', $googleUser->getId())->orWhere('email', $googleUser-
>getEmail())->first();
// Google is only for guests: staff sign in with their email and password.
if ($user && ! $user->hasRole(Role::CLIENT)) {
    return redirect()->away("{$frontend}/login?error=google_personal");
}
```

On the web, each route declares which roles can open it; otherwise the 403 screen is shown (and the API also answers 403):

web/src/auth/ProtectedRoute.tsx

```
export function ProtectedRoute({ roles }: { roles?: RoleName[] }) {
  const { user, hasRole } = useAuth()
  if (!user) return <Navigate to="/login" replace />
  if (roles && !hasRole(...roles)) return <ForbiddenPage />
  return <Outlet />
}
```

3.2 Registration screens

The “Register staff” and the guest’s “Complete registration” screens are shown with their description in sections 4.3 and 4.5.

4. What each role can do

Each role sees only its options in the menu. For each role, its menu is shown first and then each menu view separately, without the side bar, with a description of what it is for above each screenshot. When a view opens a pop-up window (form), only that window is shown. The screenshots were taken on October 27, 2026 with the page running on the project’s MySQL database (70 rooms, 1,000 guests and 5,000 demo reservations). The application interface is in Spanish; the menu option names are given in English with the Spanish label in quotes.

Menu option	Admin	Front desk	Storeroom	Developer	Guest
Dashboard («Panel»)	✓	✓	✓	—	—
Reservations, Stays, Rooms, Guests, Parking	✓	✓	—	—	—
Room types	edits	views	—	—	—
Inventory	✓	—	✓	—	—
Staff, Users and roles, Reports, Audit, Billing	✓	—	—	—	—
Register staff	✓	—	—	✓	—
Developer view	✓	—	—	✓	—
Book, My bookings	—	—	—	—	✓

If someone types the address of an option that is not in their menu, they see the 403 screen (section 5) and the API also answers 403.

4.1 Receptionist

Serves guests day to day: books for them, does check-in and check-out, and manages parking and room status.

Receptionist menu

It has the “Inicio” group with the Dashboard and the “Recepción” group with its six work views. It does not see Inventory or the Administration options.

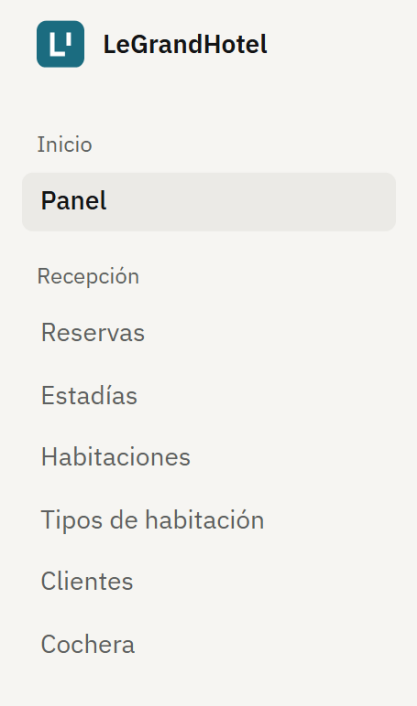


Figure 5. Receptionist menu.

Front desk dashboard («Panel»)

Summary of the day: occupancy, free rooms, today's arrivals and departures and rooms being cleaned. On the right, today's arrivals with their check-in button and the supplies below the minimum; below, the room board by floor with each room's status.

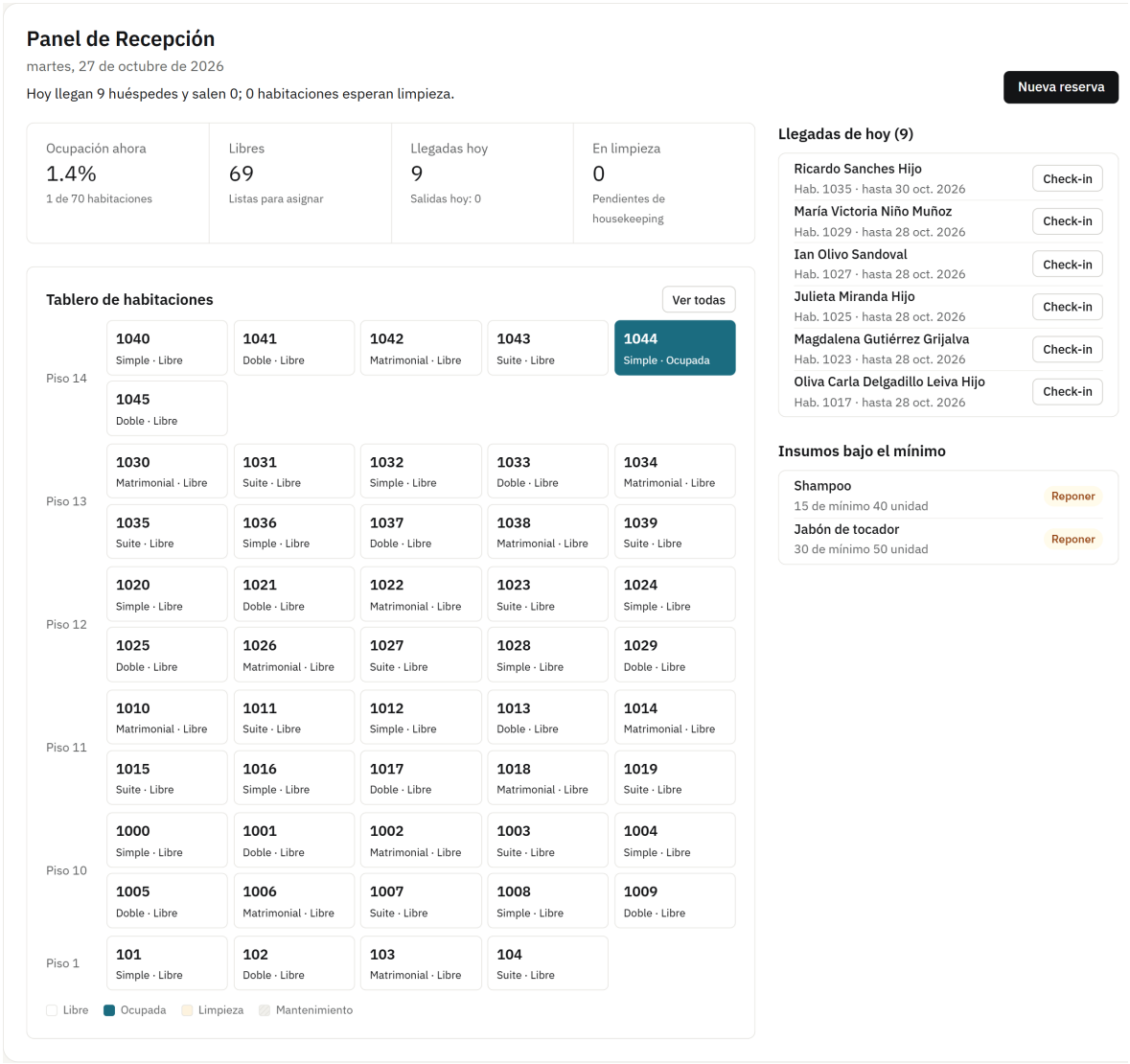


Figure 6. Front desk dashboard («Panel»)

Reservations («Reservas»)

Reservation list paginated on the server, with tabs by status (confirmed, pending, checked in, completed and cancelled) and search by guest or document. From each row you can check in or cancel the reservation.

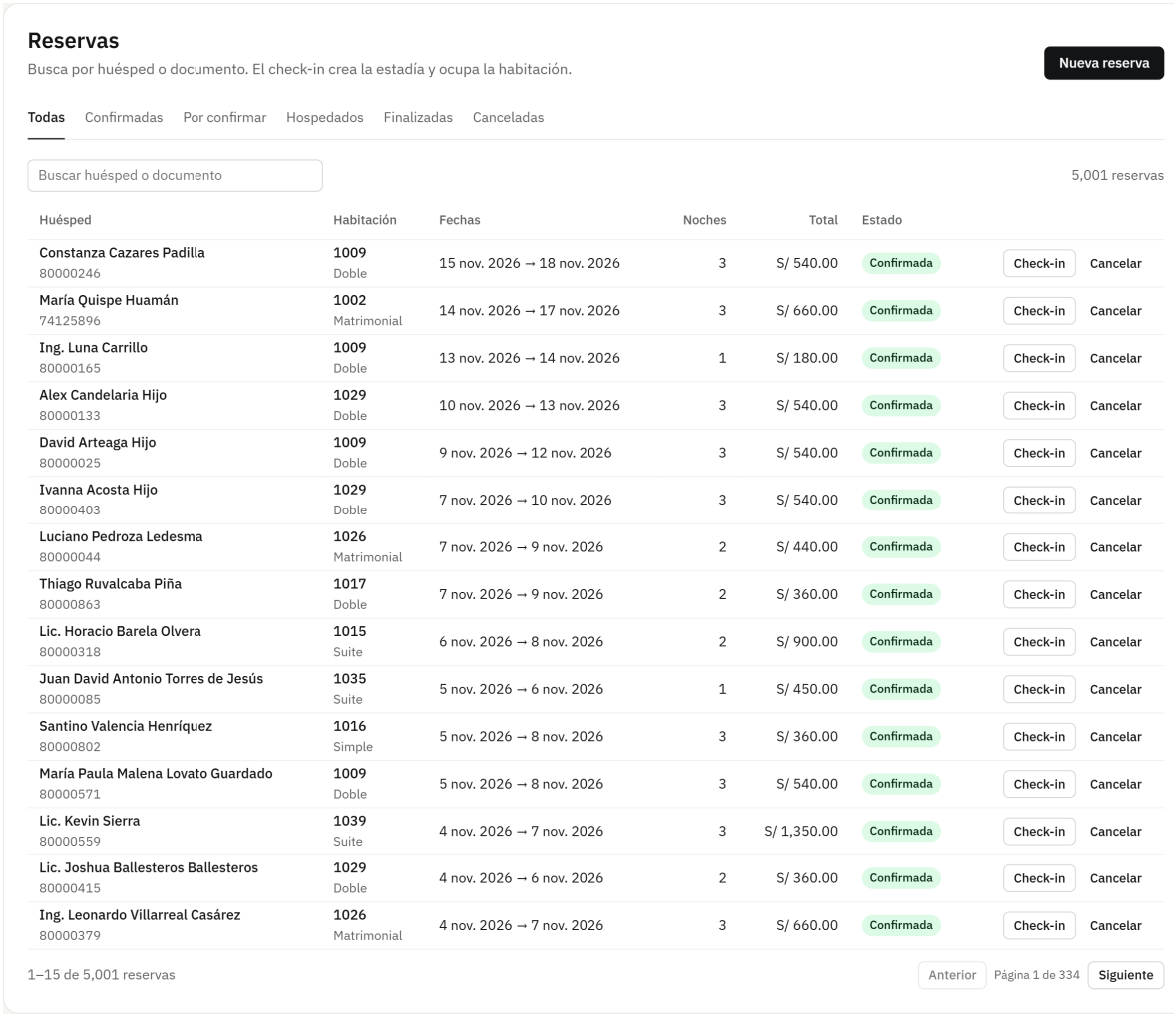


Figure 7. Reservations («Reservas»)

Pop-up window: New reservation

You search for the guest, choose the dates and number of guests, and only the rooms free for those dates appear. The server calculates the total; if someone else takes the same room at the same moment, the API answers 409.

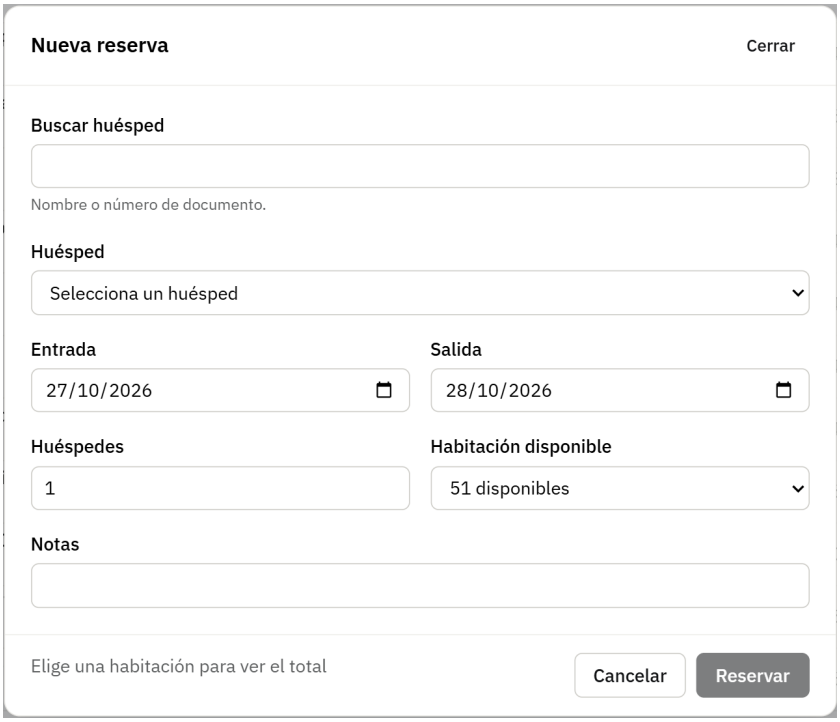


Figure 8. Pop-up window: New reservation

Stays («Estadías»)

Guests currently in the hotel and the stay history. Check-out closes the stay, sends the room to cleaning and frees its parking spot.

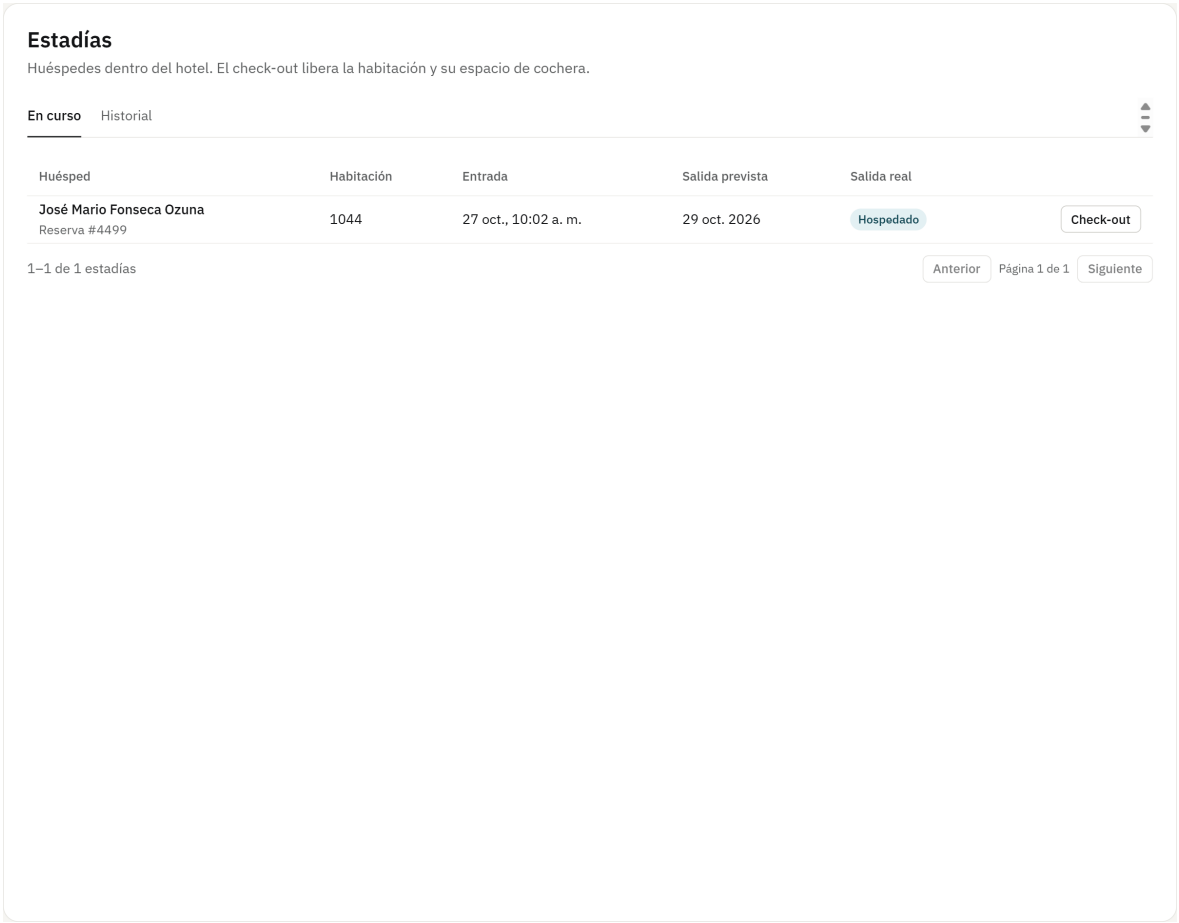


Figure 9. Stays («Estadías»)

Rooms («Habitaciones»)

Rooms with their floor, type, capacity, price and status, filterable by status. The front desk marks them as “ready” after cleaning.

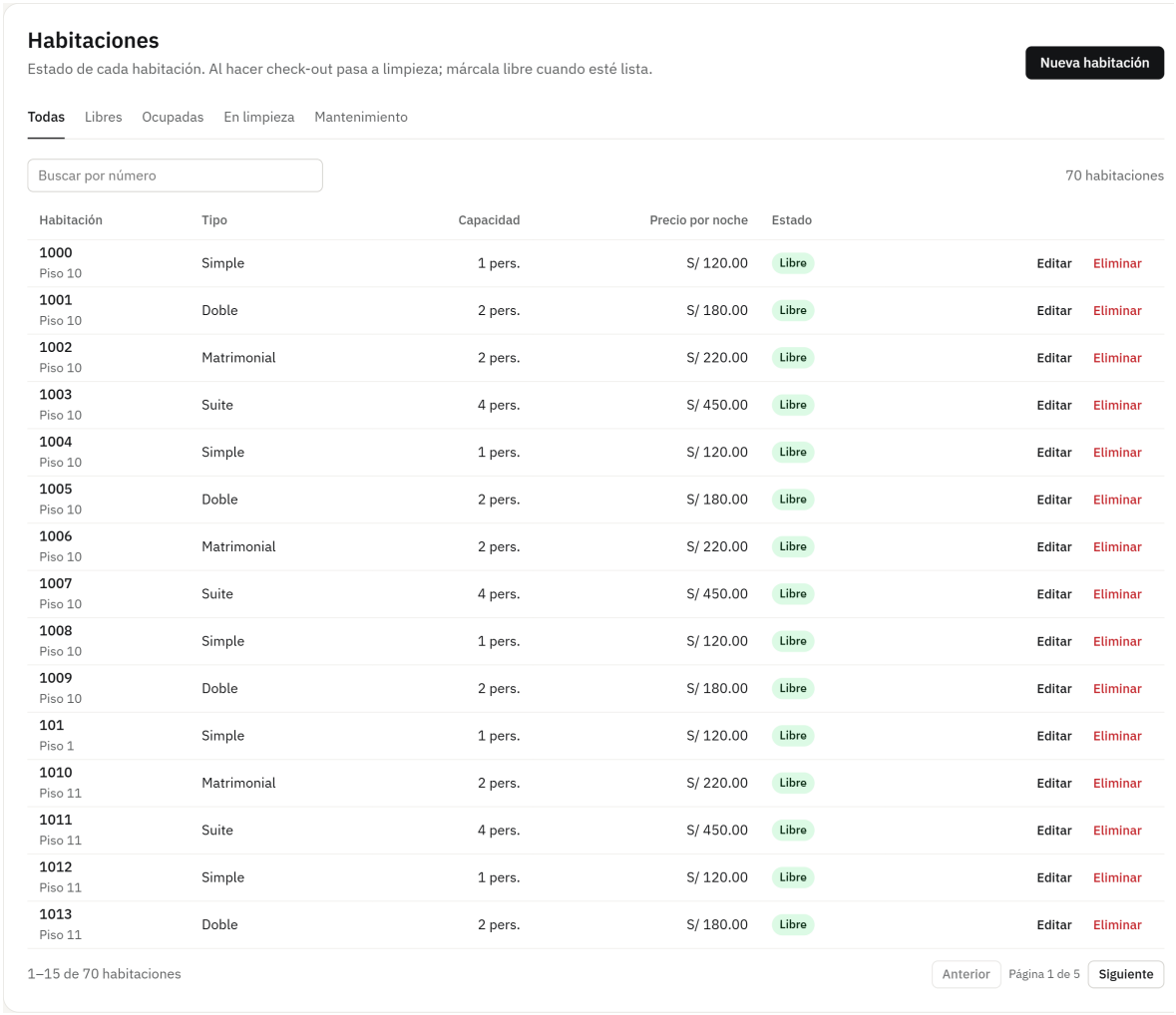


Figure 10. Rooms («Habitaciones»)

Room types (read only)

The front desk checks the price per night, capacity and amenities of each type to inform guests, but cannot create or change them.

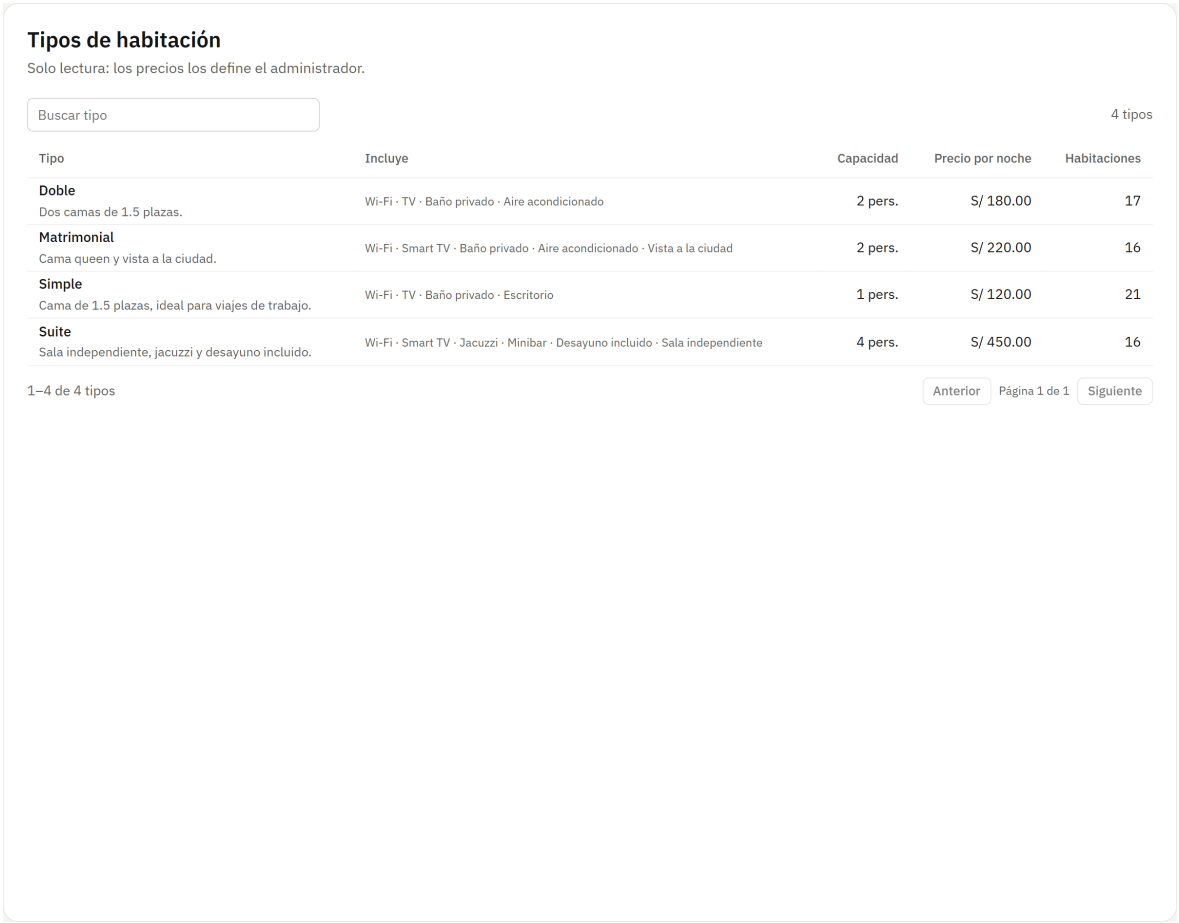


Figure 11. Room types (read only)

Guests («Clientes»)

Guest records with their document, phone, whether they have a web account and how many reservations they have. Only a guest without reservations can be deleted.

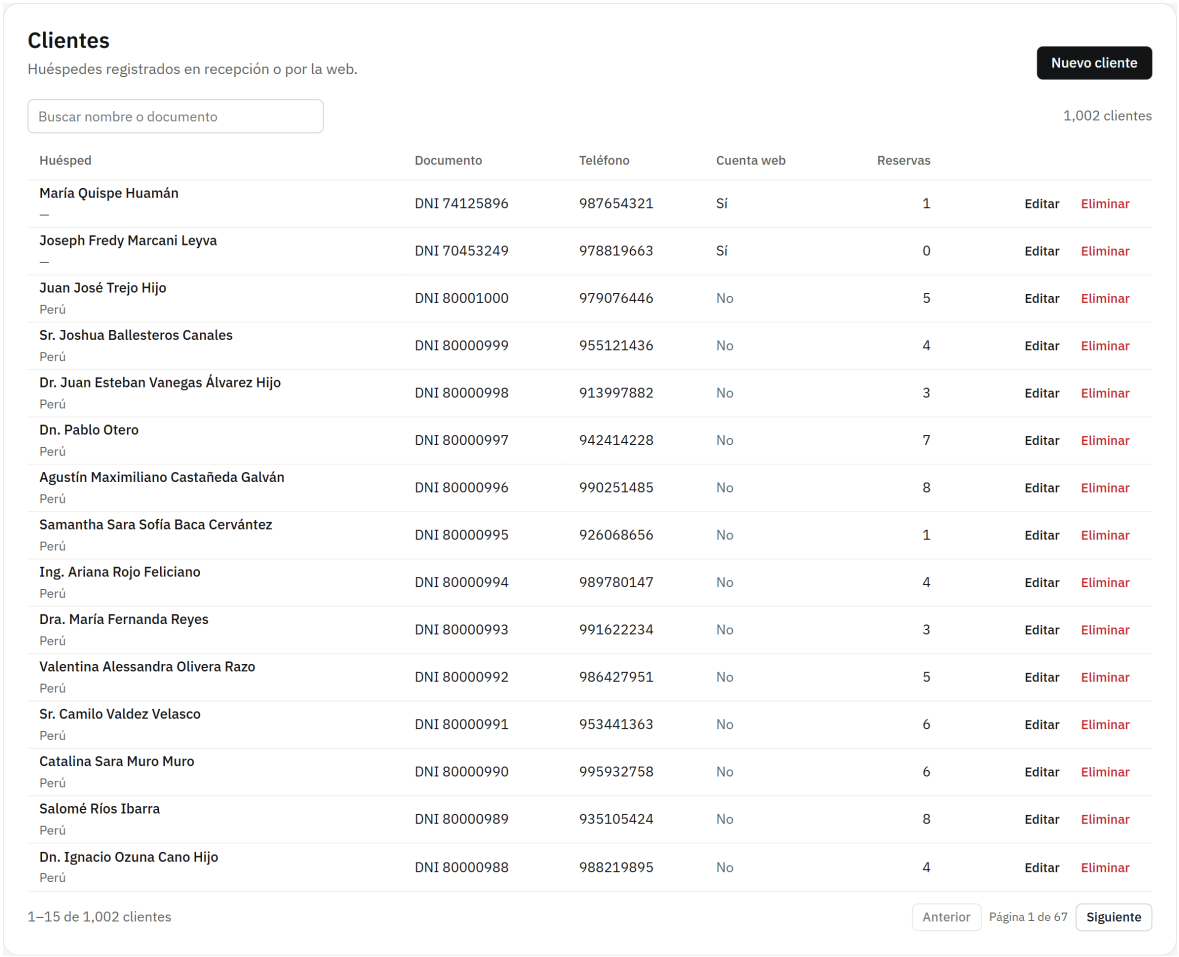


Figure 12. Guests («Clientes»)

Pop-up window: New guest

Registers a guest who arrives or calls: name, document type and number (cannot repeat), phone and nationality.

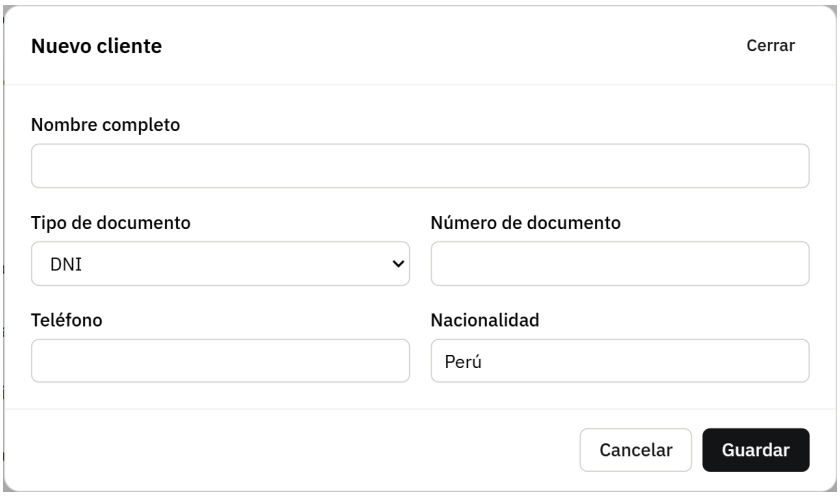


Figure 13. Pop-up window: New guest

Parking («Cochera»)

Vehicles currently inside and the parking spots with their status.

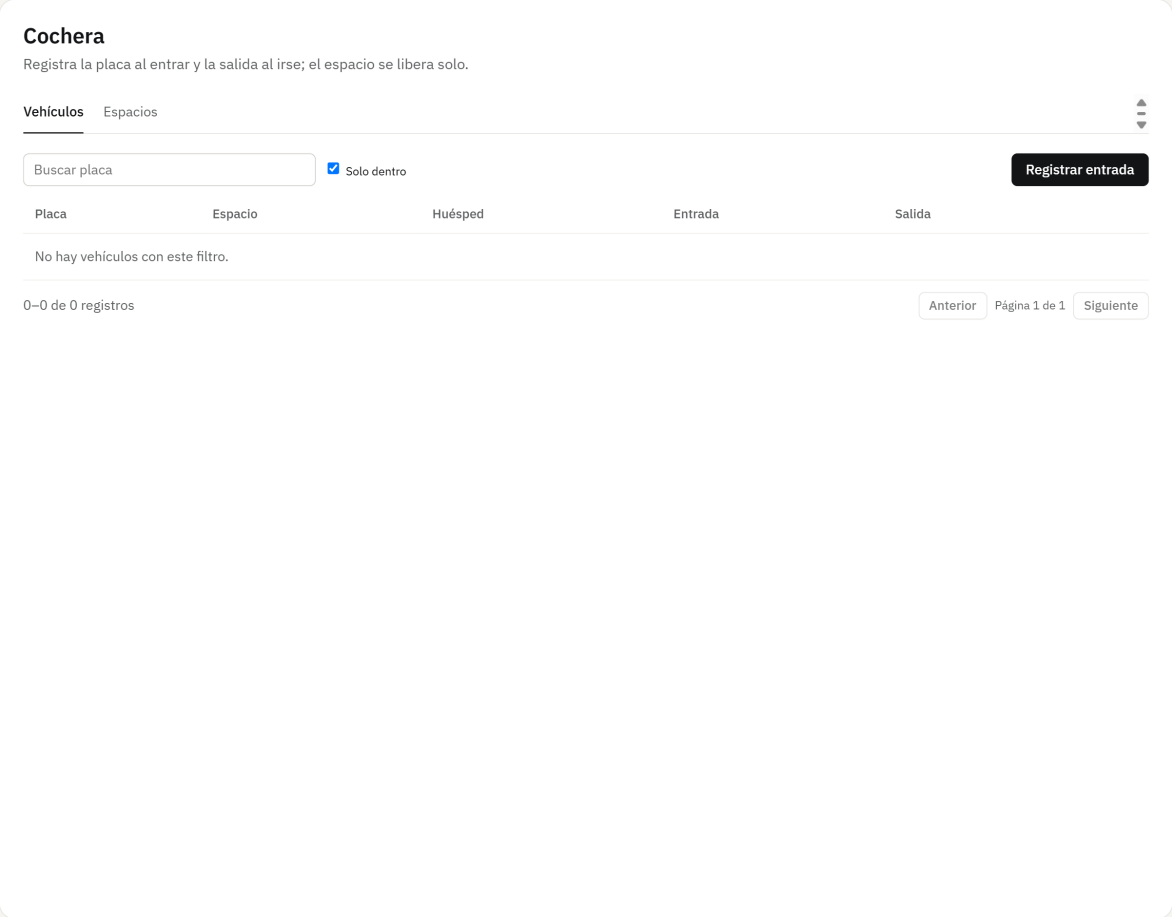


Figure 14. Parking («Cochera»)

Pop-up window: Record parking entry

You enter the plate, the free spot and, if applicable, the guest's stay. If the spot is already taken, the API answers 409 PARKING_SPOT_NOT_FREE.

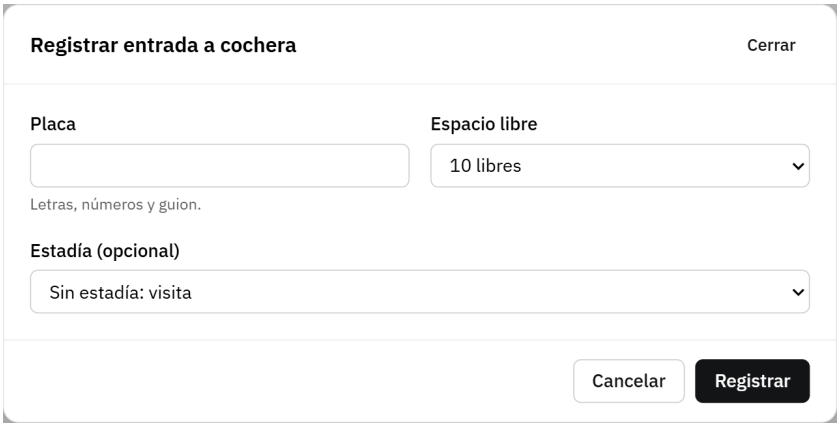


Figure 15. Pop-up window: Record parking entry

4.2 Storekeeper

Manages the hotel's supplies: what comes in, what goes out and what needs restocking.

Storekeeper menu

It has only two options: the Dashboard and Inventory.

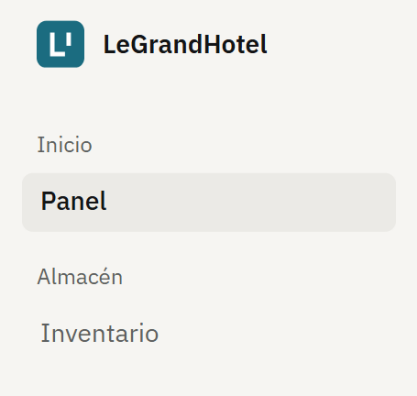


Figure 16. Storekeeper menu.

Storeroom dashboard («Panel»)

Shows the supplies below the minimum, to restock them in time, and the button to record a movement.

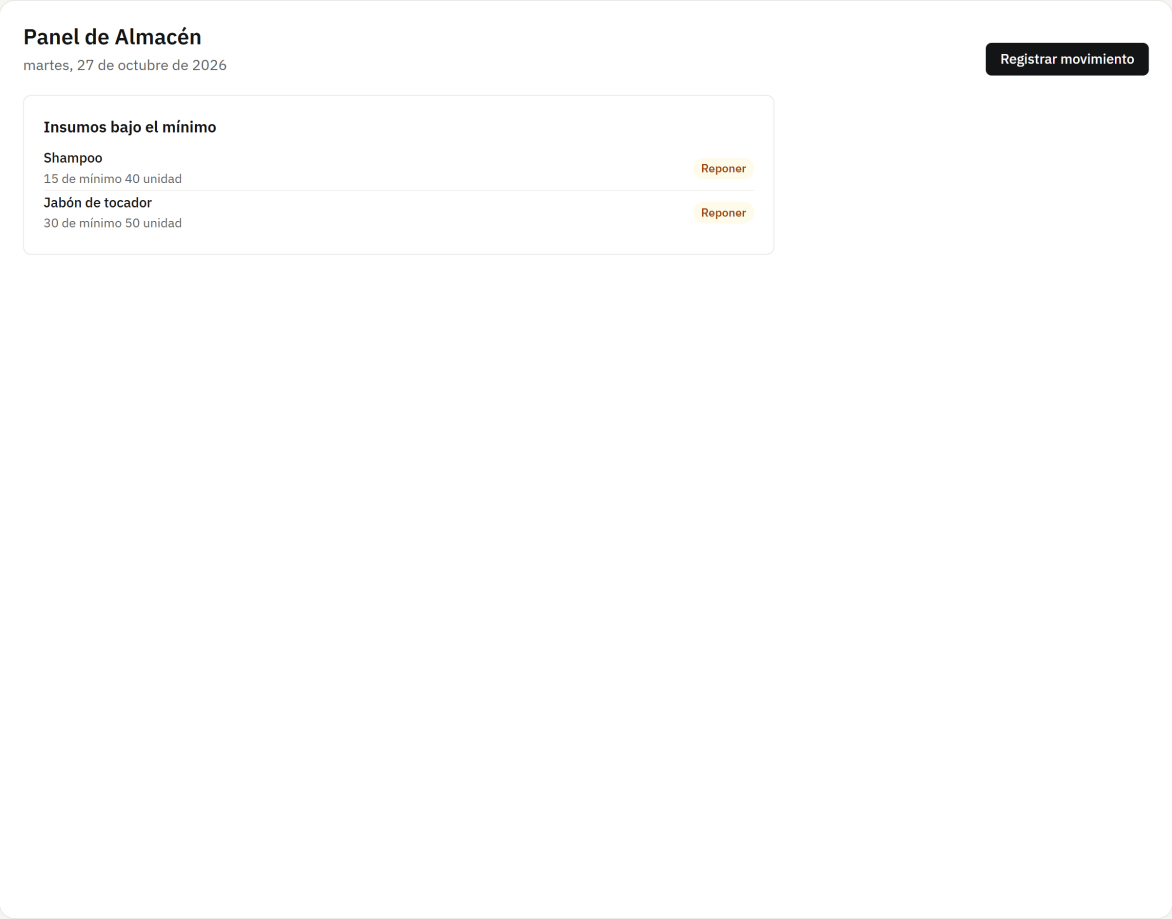


Figure 17. Storeroom dashboard («Panel»)

Inventory («Inventario»)

Each supply with its unit, stock, minimum and a level bar; you can show only those below the minimum. From each row you record a movement or edit the supply.

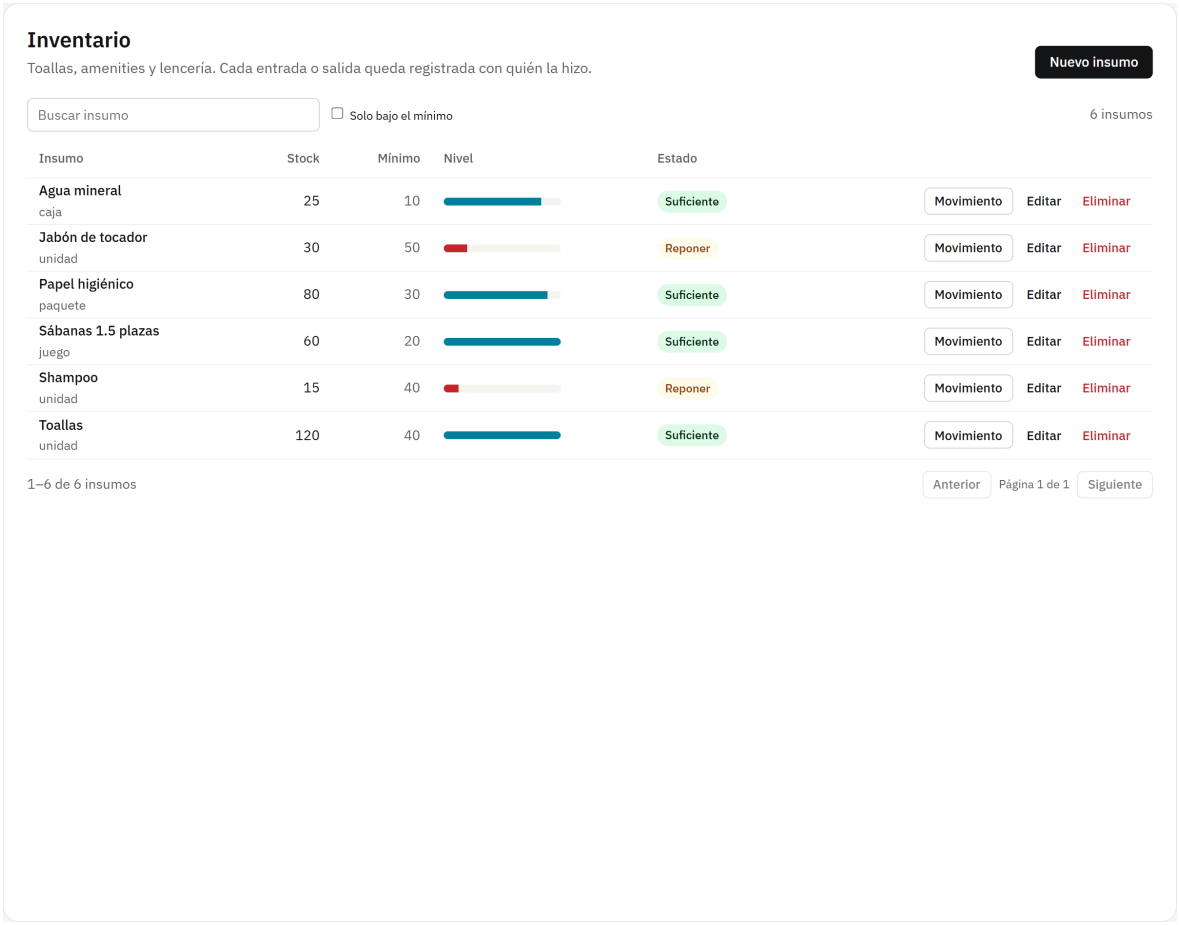


Figure 18. Inventory («Inventario»)

Pop-up window: Record movement

Stock in or out with quantity and reason; the latest movements of the supply appear below. The API does not allow taking out more than there is (409 INSUFFICIENT_STOCK).

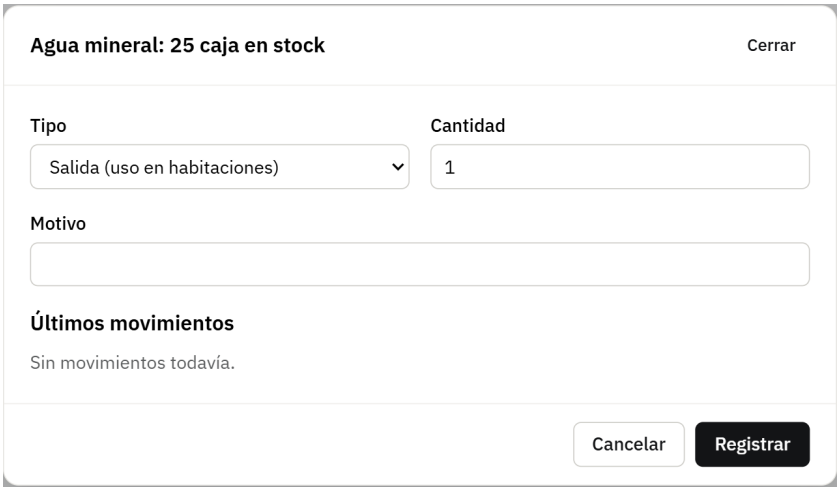


Figure 19. Pop-up window: Record movement

Pop-up window: New supply

Name, unit, initial stock and minimum. After that, stock only changes through movements, so everything is recorded.

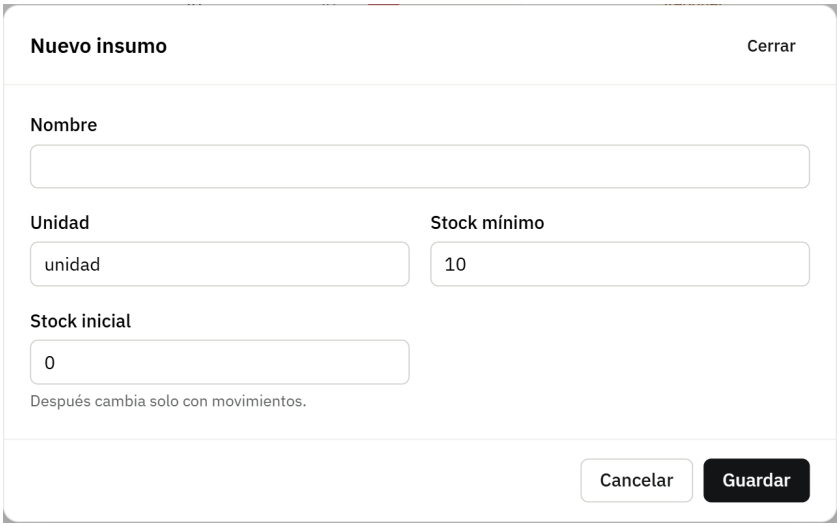


Figure 20. Pop-up window: New supply

4.3 Administrator

Responsible for the system: has every staff option, defines room types and prices, manages staff and their accounts, and reviews reports and the audit log.

Administrator menu

It is the full menu: Inicio, Recepción, Almacén, Administración and Programador.

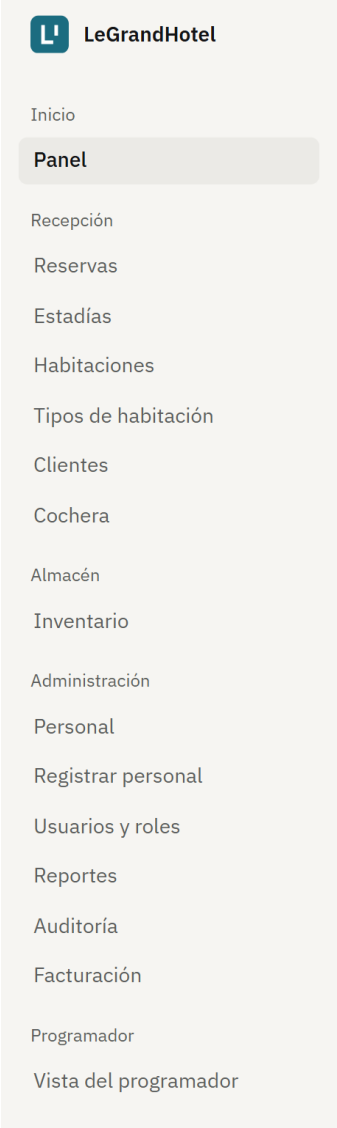


Figure 21. Administrator menu.

Administrator dashboard («Panel»)

The same daily summary as the front desk, plus the month’s income.

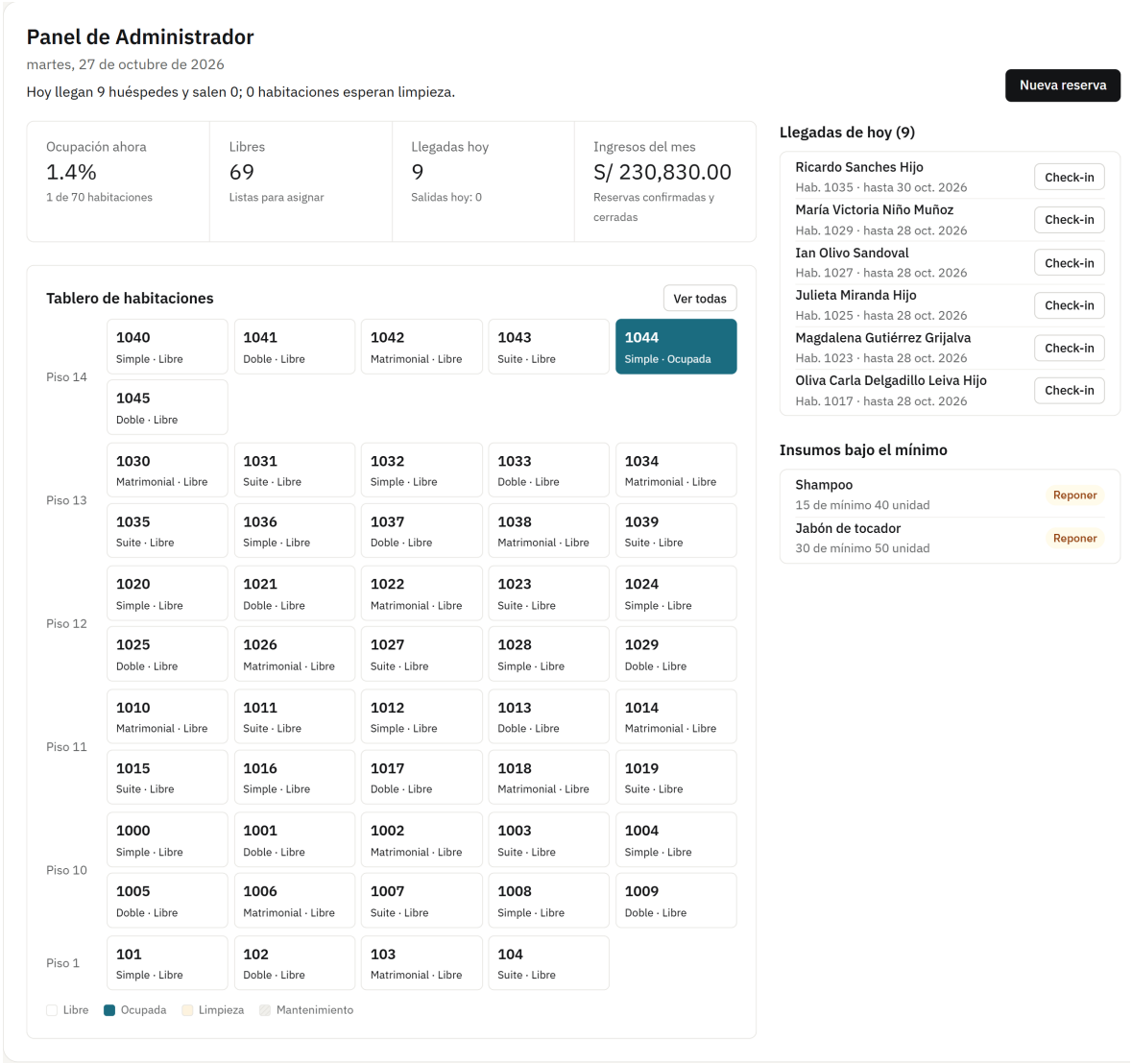


Figure 22. Administrator dashboard («Panel»)

Reservations

The same reservations the front desk manages: the Administrator can also create, check in and cancel.

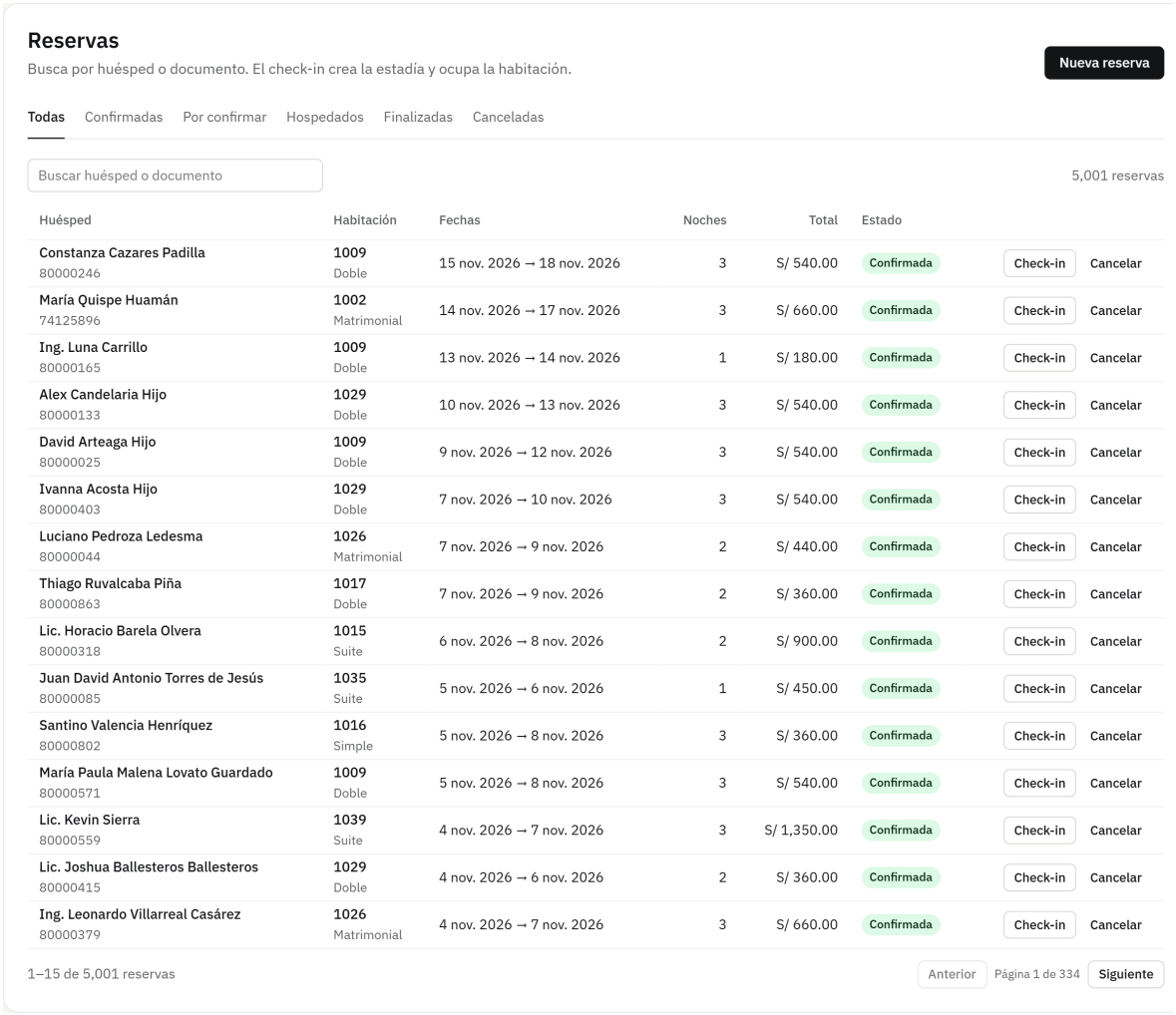


Figure 23. Reservations

Stays

Guests in the hotel and the history, with check-out.

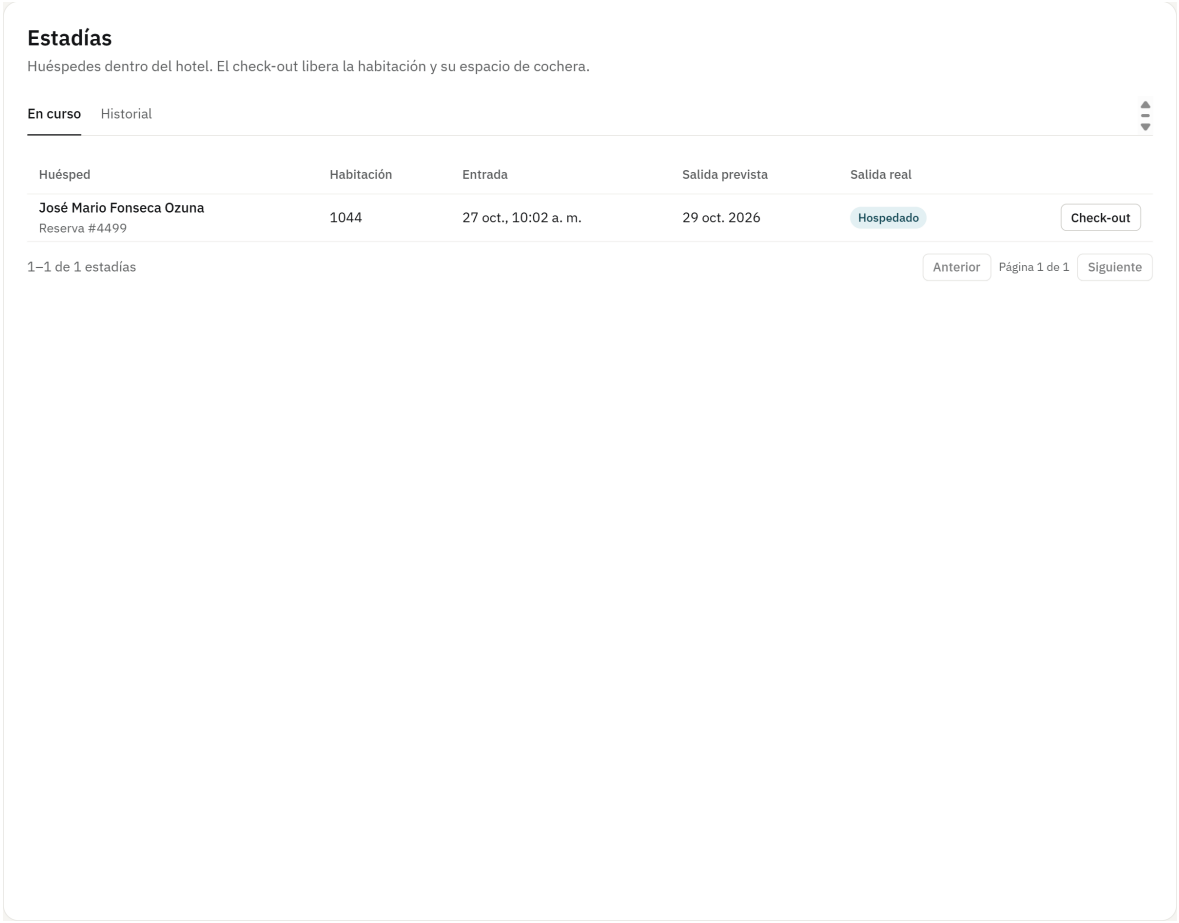


Figure 24. Stays

Rooms

Rooms by status; the Administrator also creates and edits rooms.

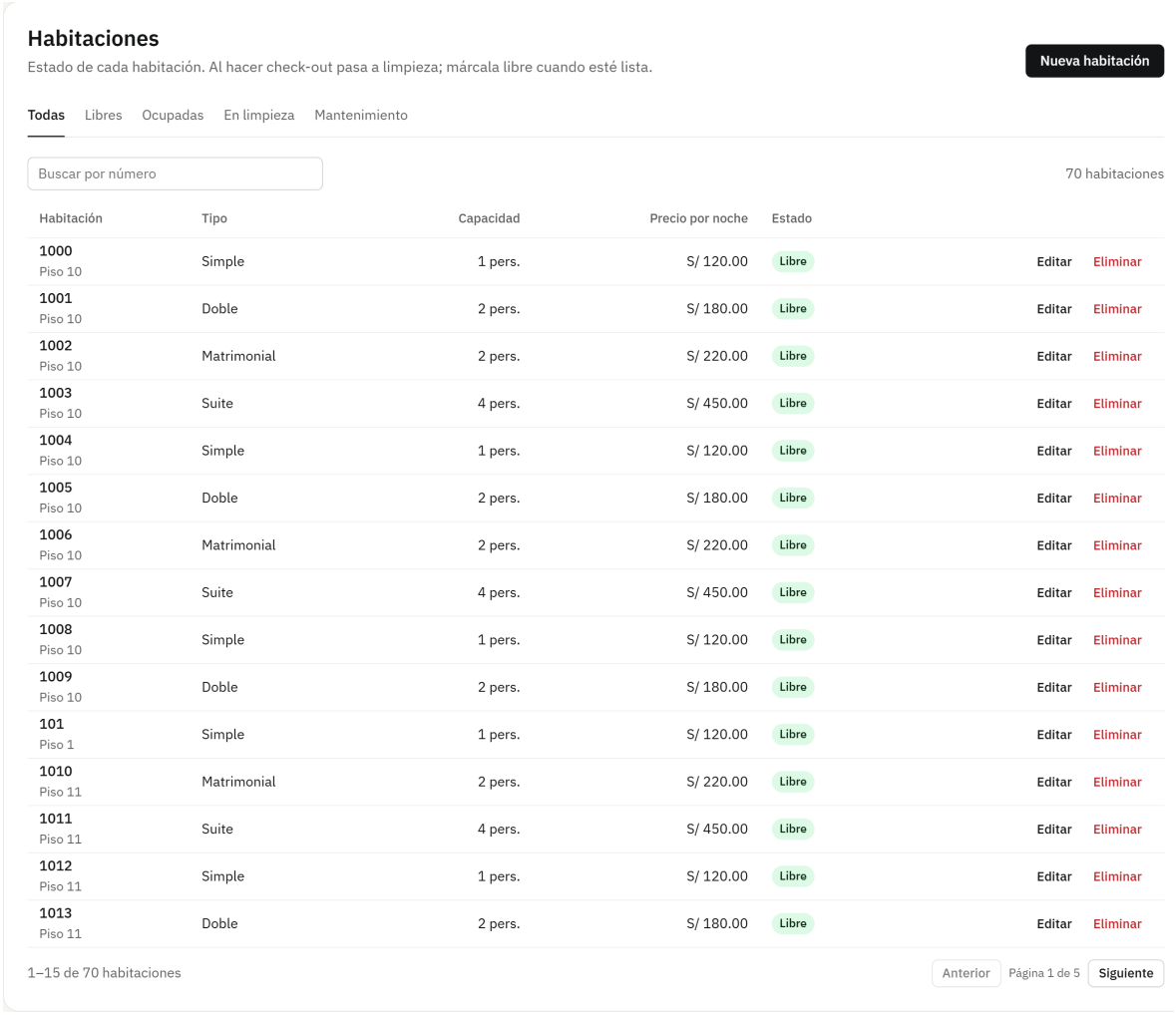


Figure 25. Rooms

Pop-up window: New room

Number (unique), floor, type and initial status of the room.

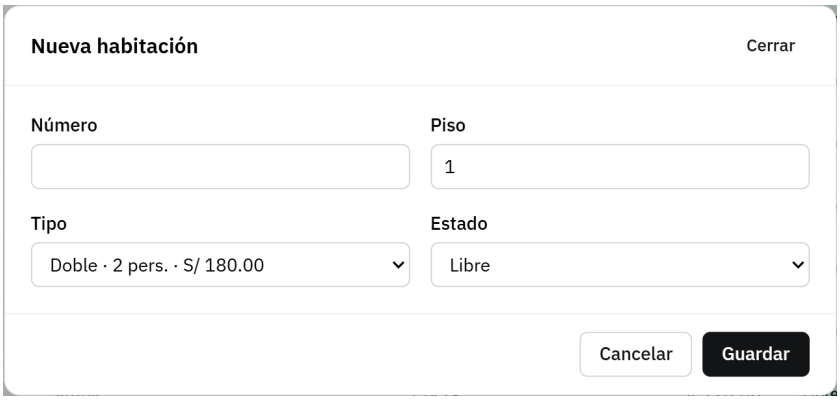


Figure 26. Pop-up window: New room

Room types

Unlike the front desk, the Administrator creates, edits and deletes room types.

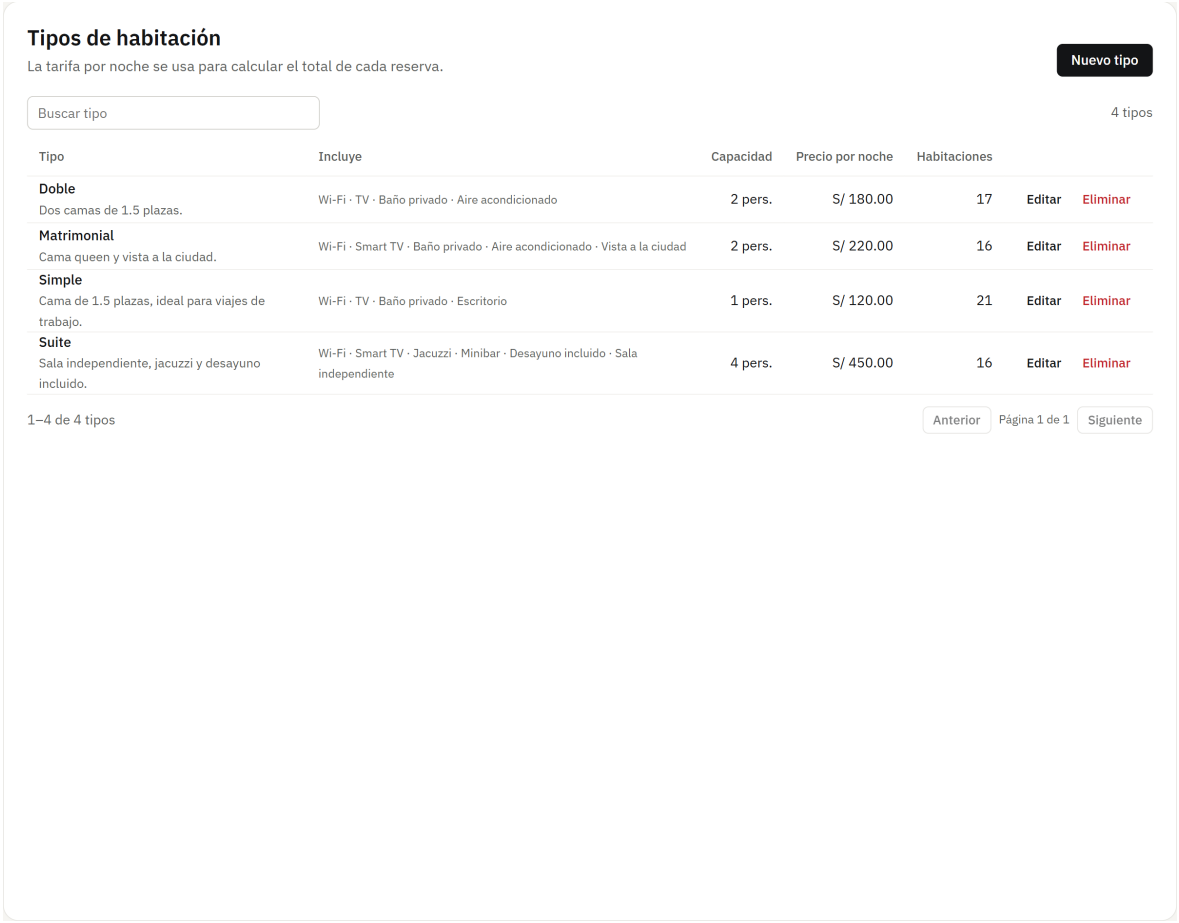


Figure 27. Room types

Pop-up window: New room type

Name, capacity, price per night, description and amenities, comma-separated. This is what the guest sees when booking.

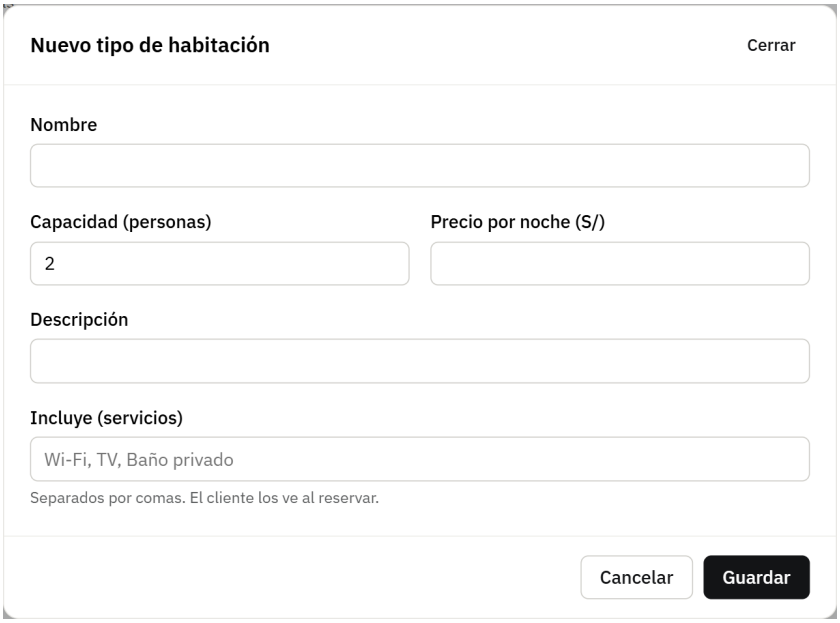


Figure 28. Pop-up window: New room type

Guests

The guest records, same as the front desk.

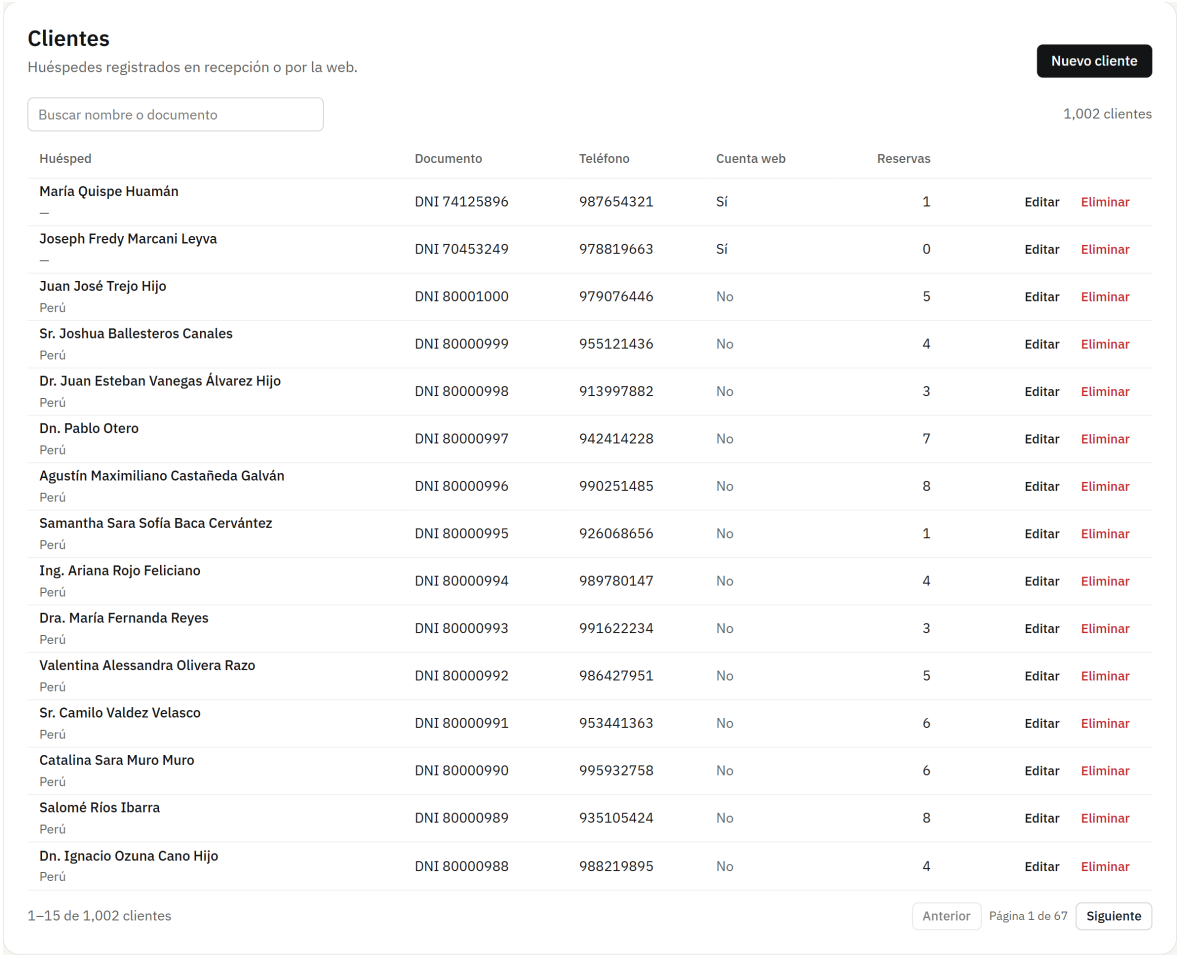


Figure 29. Guests

Parking

Vehicles and spots; the Administrator also adds spots.

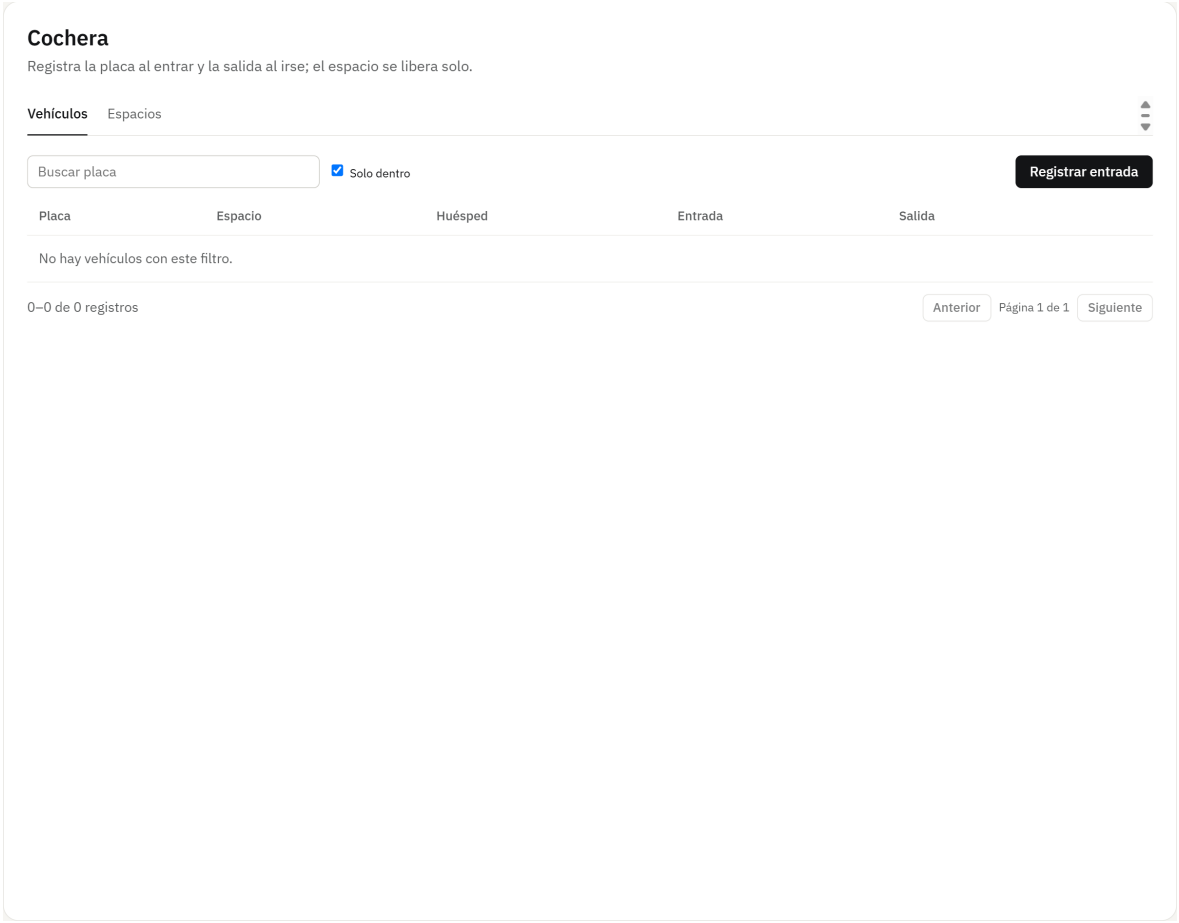


Figure 30. Parking

Inventory

The same inventory as the Storekeeper, with the same permissions.

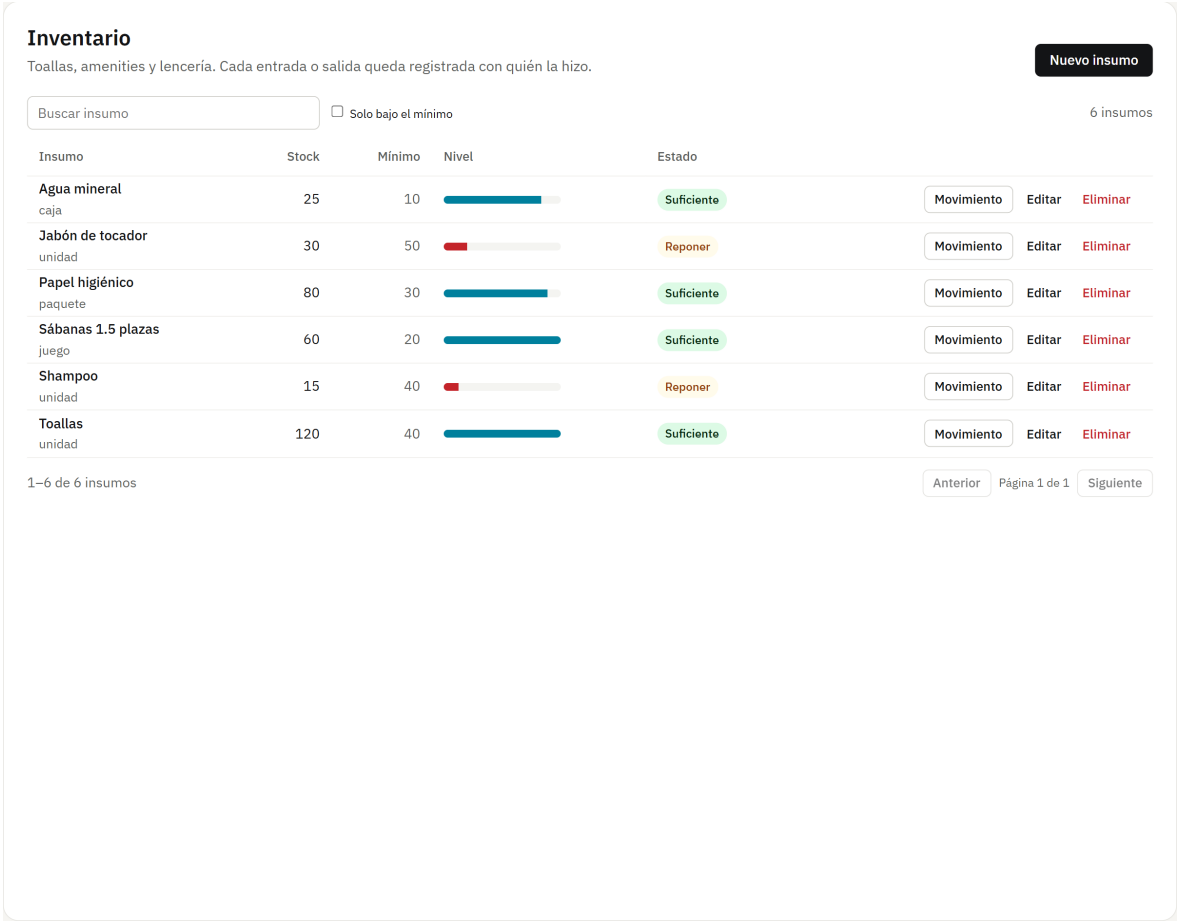


Figure 31. Inventory

Staff («Personal»)

Staff records with position, shift and start date, each linked to its account.

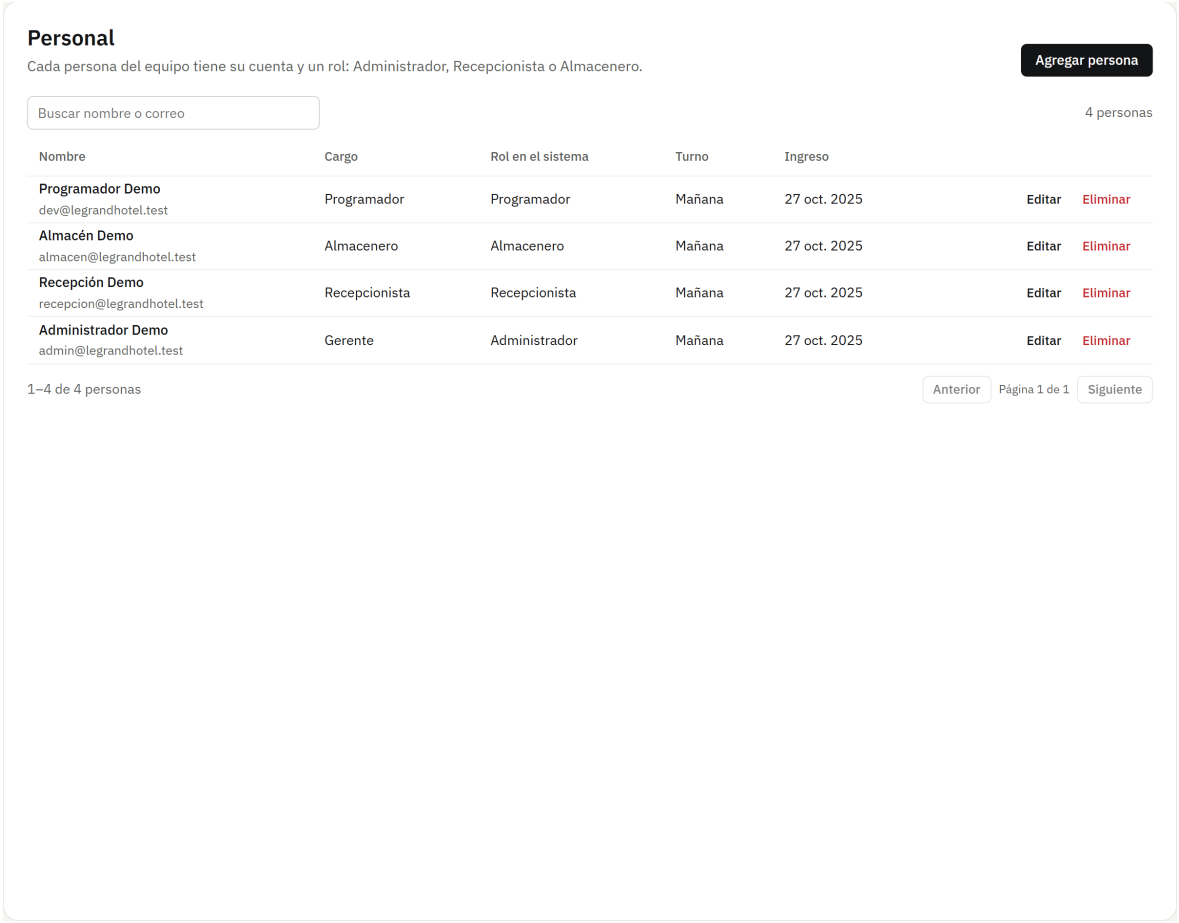


Figure 32. Staff («Personal»)

Pop-up window: Add staff member

Creates the record and the account in one step. The Developer role does not appear because the hotel has only one.

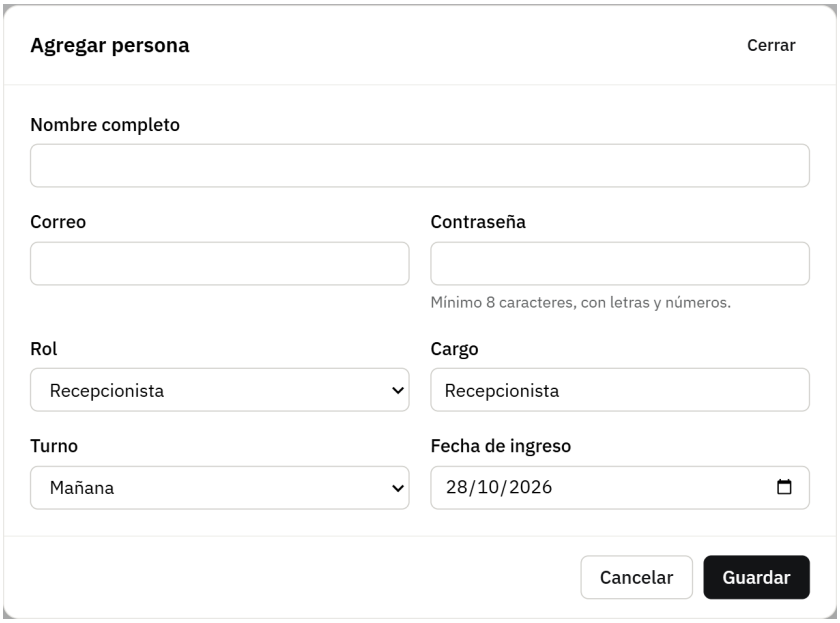


Figure 33. Pop-up window: Add staff member

Register staff («Registrar personal»)

Creates the account of a hotel staff member with an initial password. It only offers Receptionist, Storekeeper and Administrator; the new account receives an email to verify it.

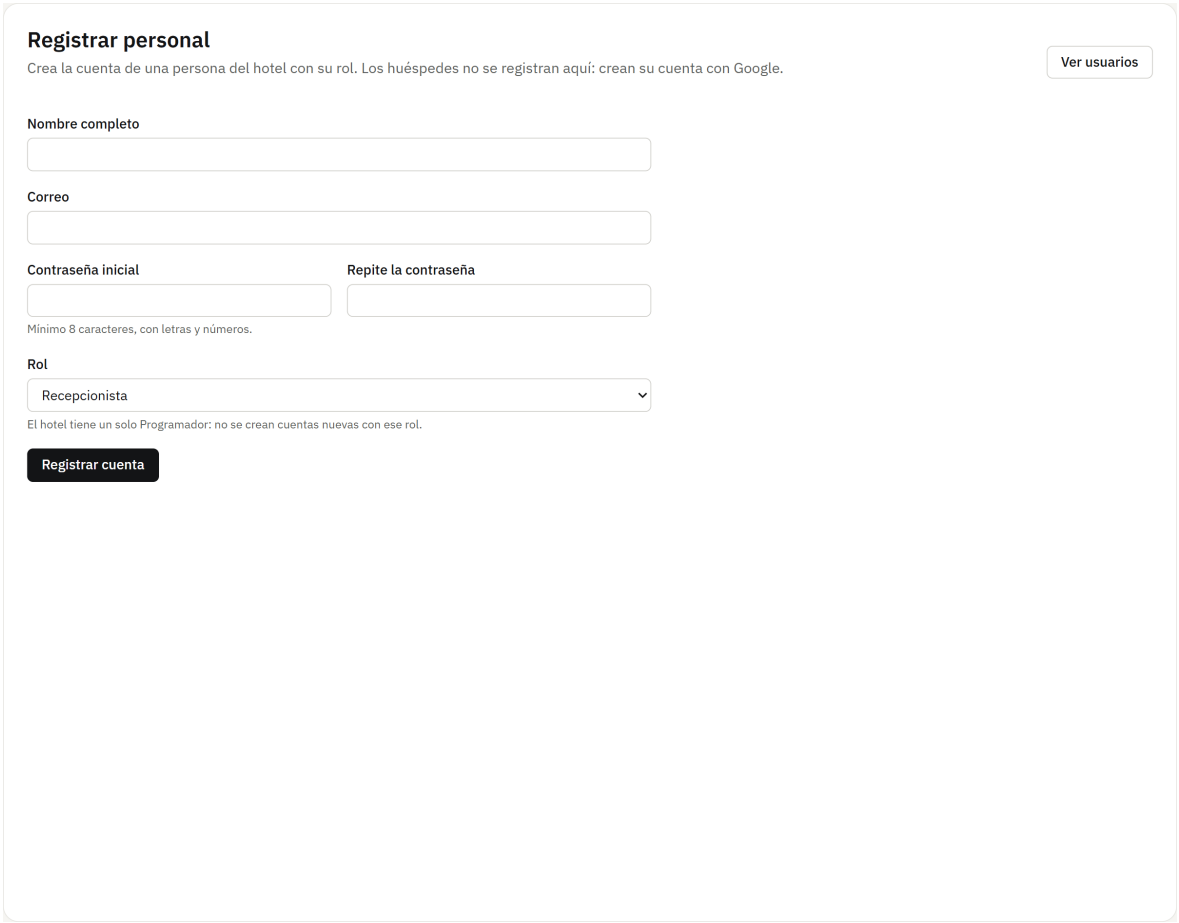


Figure 34. Register staff («Registrar personal»)

Users and roles («Usuarios y roles»)

Every account with its role and whether the email was verified, with role filters and search by name or email.

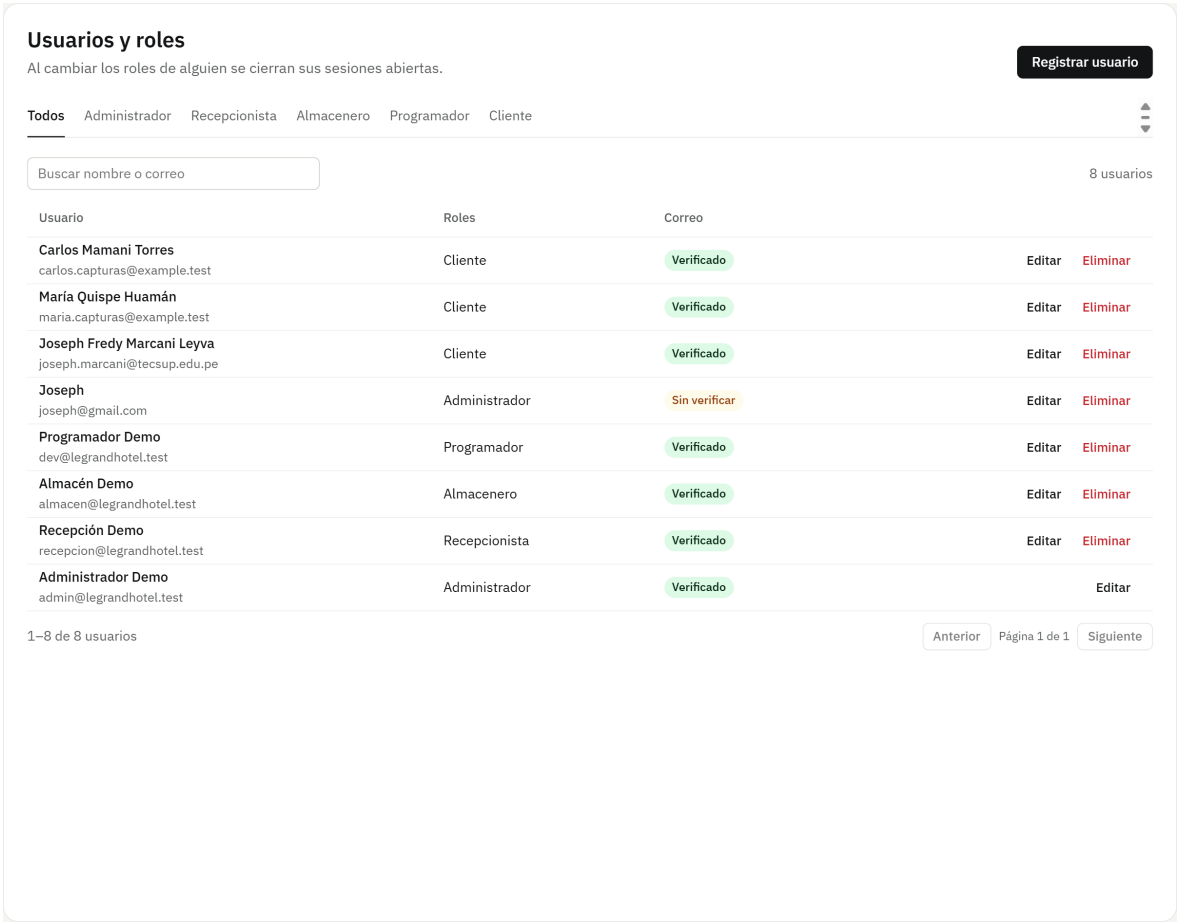


Figure 35. Users and roles («Usuarios y roles»)

Pop-up window: Edit the Developer account

The Developer checkbox is locked: the role cannot be given to another account or removed from the Developer. Their name, email and password can be changed.

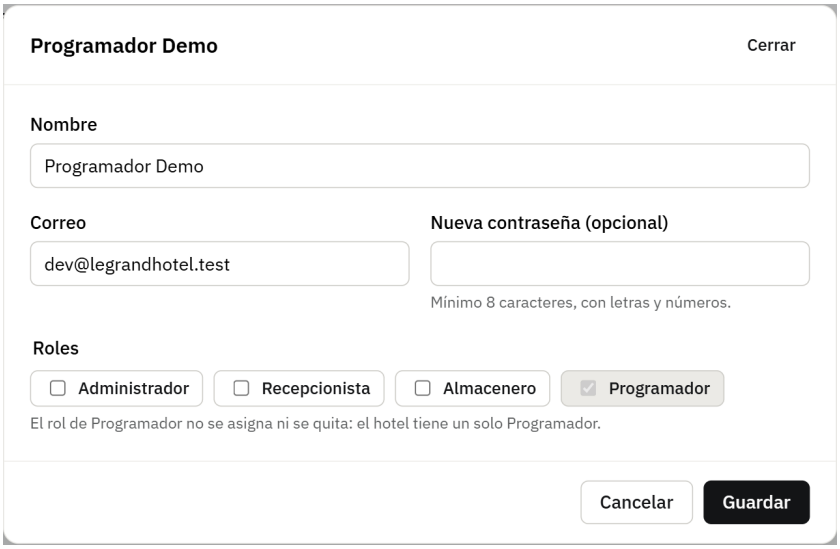


Figure 36. Pop-up window: Edit the Developer account

Reports («Reportes»)

Occupancy for a date range and income for the last 6 months in a bar chart.

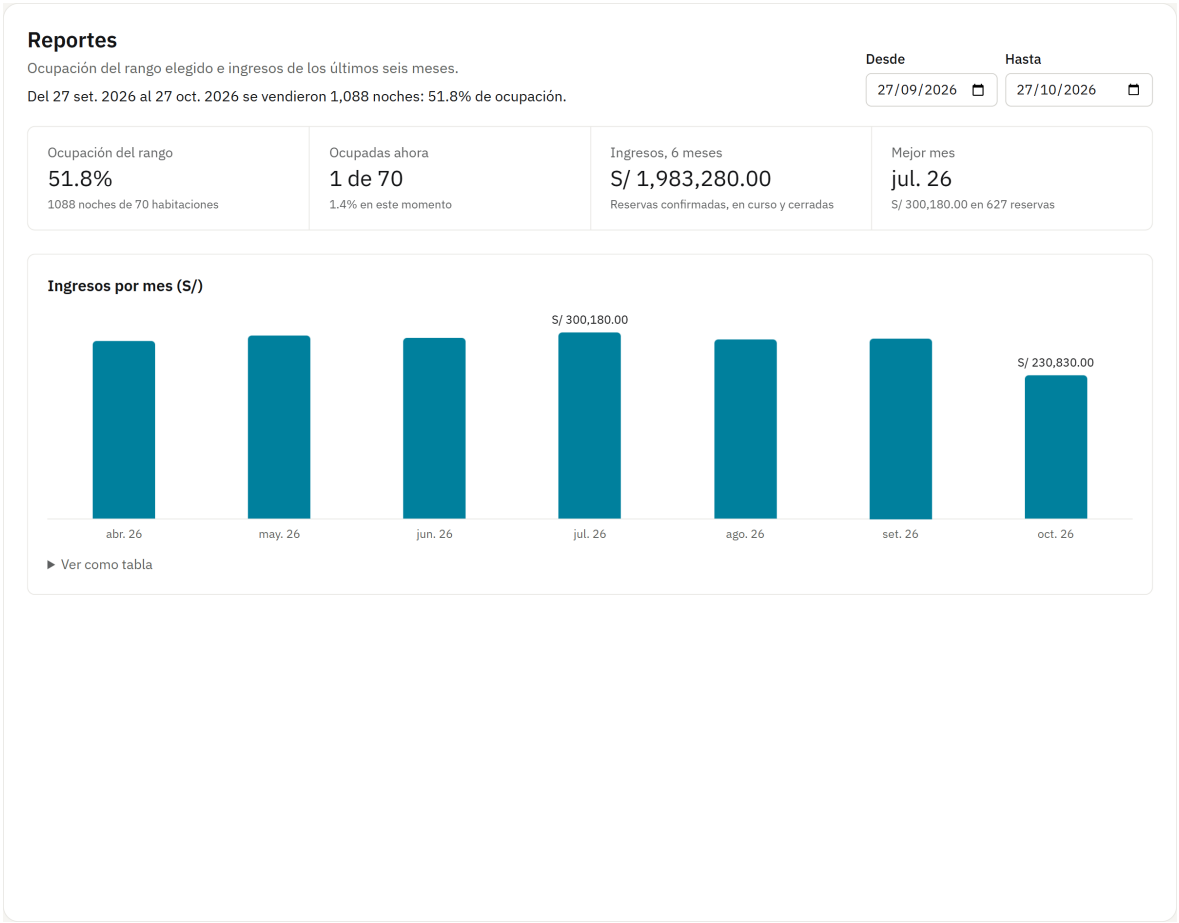
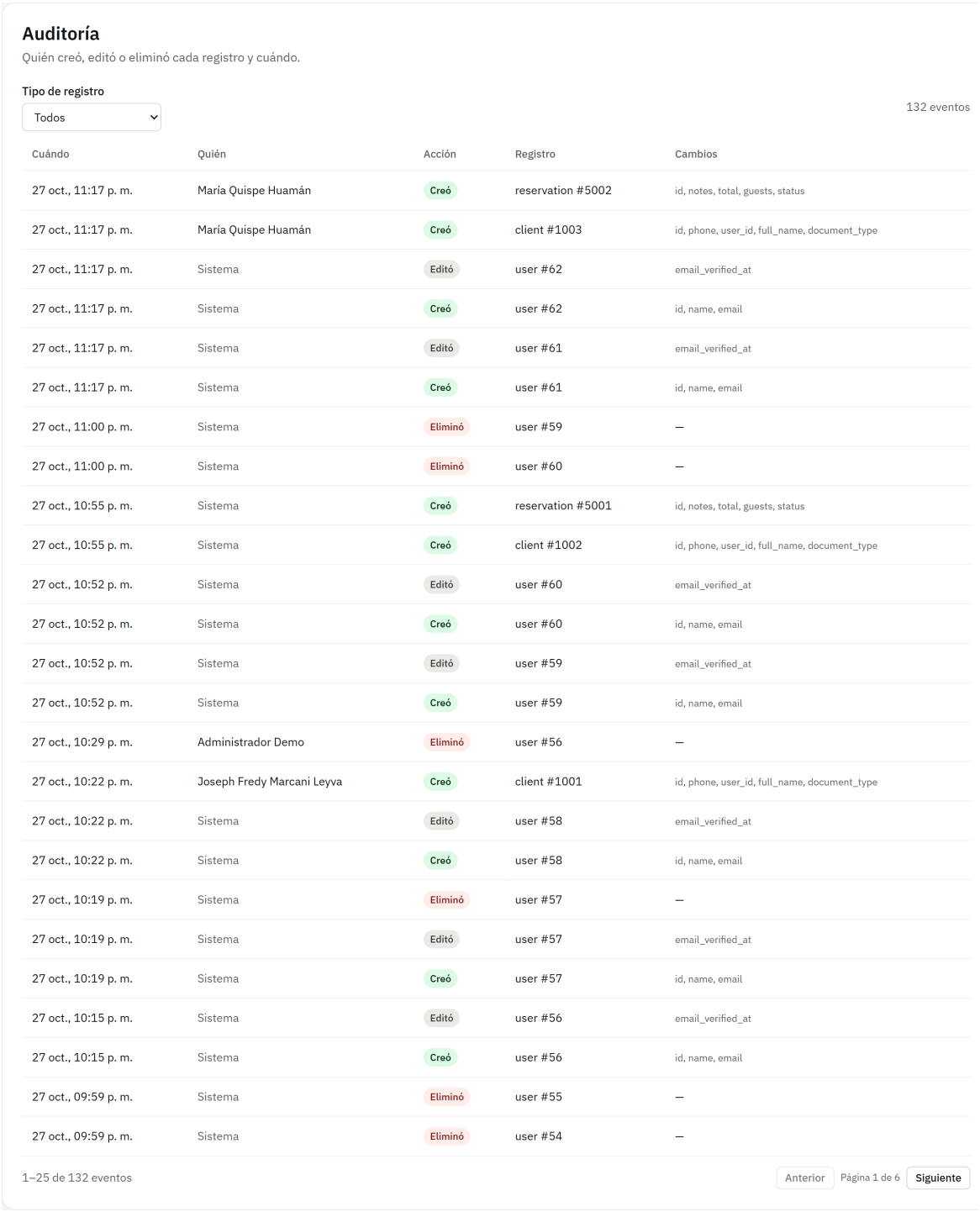


Figure 37. Reports («Reportes»)

Audit («Auditoría»)

Who created, edited or deleted each record, when and from which IP, with the values before and after the change.



Auditoría

Quién creó, editó o eliminó cada registro y cuándo.

Tipo de registro: Todos — 132 eventos

Cuándo	Quién	Acción	Registro	Cambios
27 oct., 11:17 p. m.	María Quispe Huamán	Creó	reservation #5002	id, notes, total, guests, status
27 oct., 11:17 p. m.	María Quispe Huamán	Creó	client #1003	id, phone, user_id, full_name, document_type
27 oct., 11:17 p. m.	Sistema	Editó	user #62	email_verified_at
27 oct., 11:17 p. m.	Sistema	Creó	user #62	id, name, email
27 oct., 11:17 p. m.	Sistema	Editó	user #61	email_verified_at
27 oct., 11:17 p. m.	Sistema	Creó	user #61	id, name, email
27 oct., 11:00 p. m.	Sistema	Eliminó	user #59	—
27 oct., 11:00 p. m.	Sistema	Eliminó	user #60	—
27 oct., 10:55 p. m.	Sistema	Creó	reservation #5001	id, notes, total, guests, status
27 oct., 10:55 p. m.	Sistema	Creó	client #1002	id, phone, user_id, full_name, document_type
27 oct., 10:52 p. m.	Sistema	Editó	user #60	email_verified_at
27 oct., 10:52 p. m.	Sistema	Creó	user #60	id, name, email
27 oct., 10:52 p. m.	Sistema	Editó	user #59	email_verified_at
27 oct., 10:52 p. m.	Sistema	Creó	user #59	id, name, email
27 oct., 10:29 p. m.	Administrador Demo	Eliminó	user #56	—
27 oct., 10:22 p. m.	Joseph Fredy Marcani Leyva	Creó	client #1001	id, phone, user_id, full_name, document_type
27 oct., 10:22 p. m.	Sistema	Editó	user #58	email_verified_at
27 oct., 10:22 p. m.	Sistema	Creó	user #58	id, name, email
27 oct., 10:19 p. m.	Sistema	Eliminó	user #57	—
27 oct., 10:19 p. m.	Sistema	Editó	user #57	email_verified_at
27 oct., 10:19 p. m.	Sistema	Creó	user #57	id, name, email
27 oct., 10:15 p. m.	Sistema	Editó	user #56	email_verified_at
27 oct., 10:15 p. m.	Sistema	Creó	user #56	id, name, email
27 oct., 09:59 p. m.	Sistema	Eliminó	user #55	—
27 oct., 09:59 p. m.	Sistema	Eliminó	user #54	—

1–25 de 132 eventos — Anterior — Página 1 de 6 — Siguiente

Figure 38. Audit («Auditoría»)

Billing («Facturación»)

A planned module that is not built yet: it shows “Under construction” instead of an error.



Figure 39. Billing («Facturación»)

Developer view

The Administrator also sees the API diagnostics (details in section 4.4).

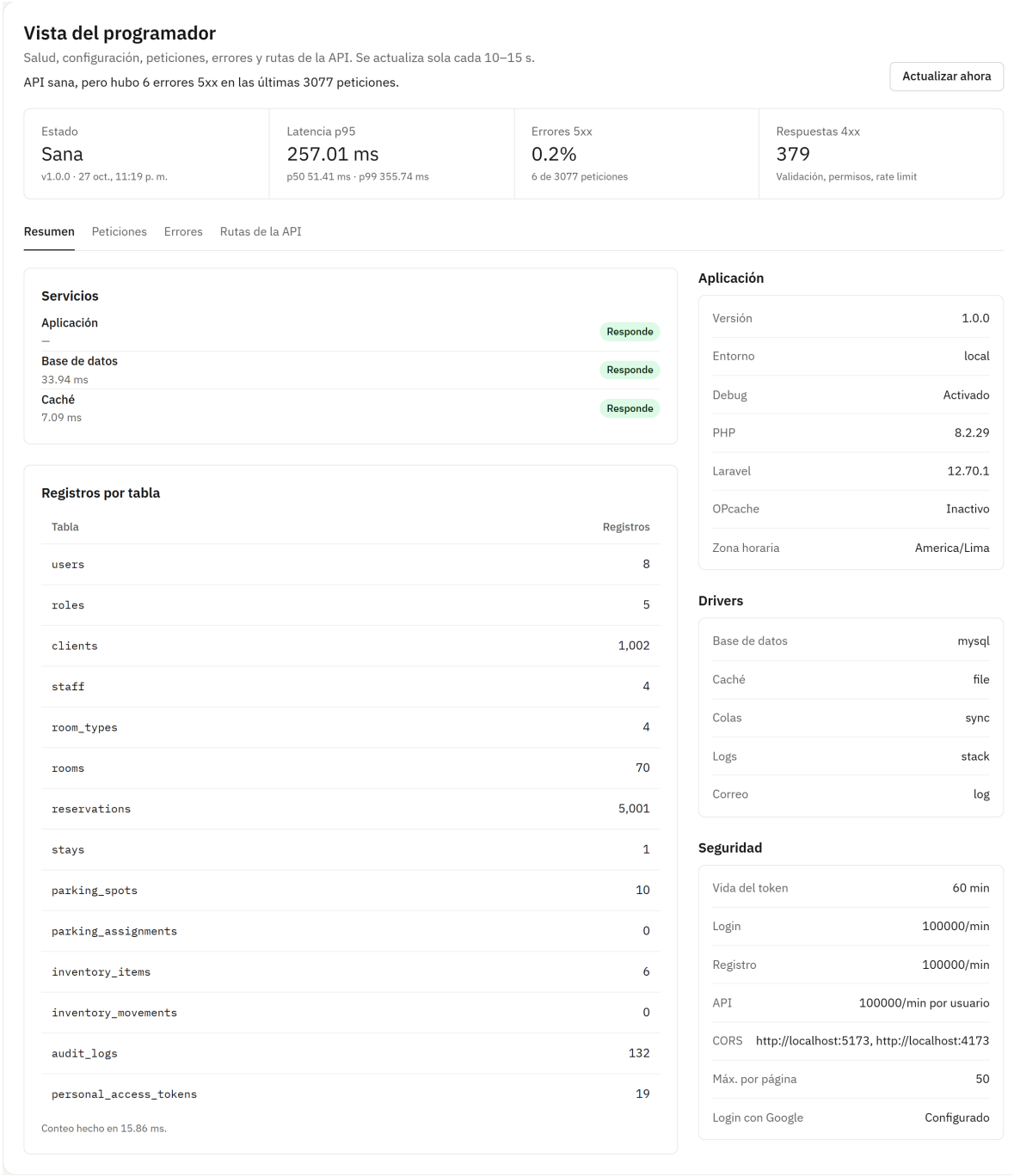


Figure 40. Developer view

4.4 Developer

Provides technical support: checks the API status without logging into the server and registers staff accounts when needed.

Developer menu

It has two options: Register staff and the Developer view.

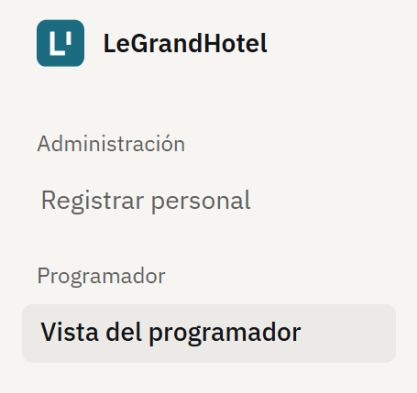


Figure 41. Developer menu.

Register staff

Like the Administrator, creates Front desk, Storeroom and Administration accounts, but not Developer accounts.

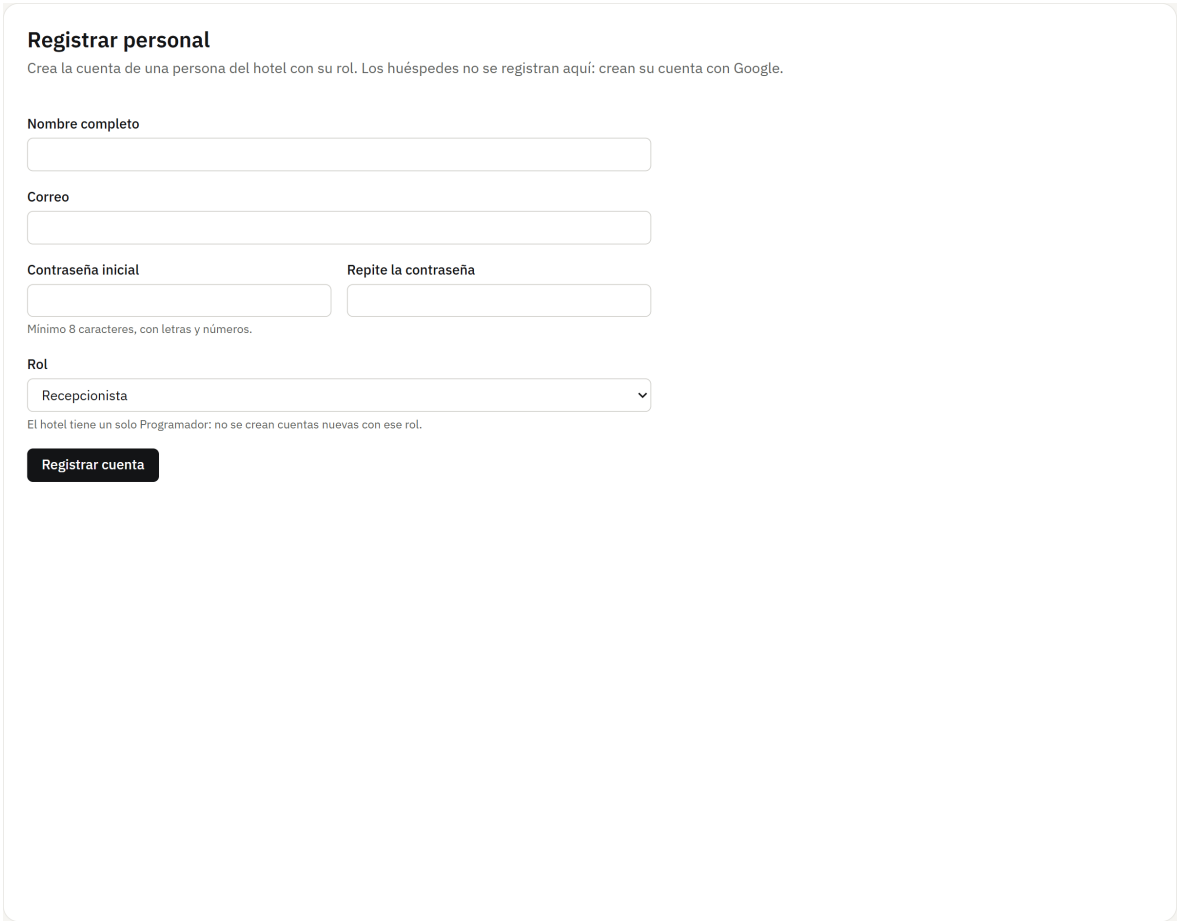


Figure 42. Register staff

Developer view · Summary

API status, p50, p95 and p99 latencies, percentage of 5xx errors and 4xx responses, and the configuration without showing secrets. It refreshes by itself every 10 to 15 seconds.

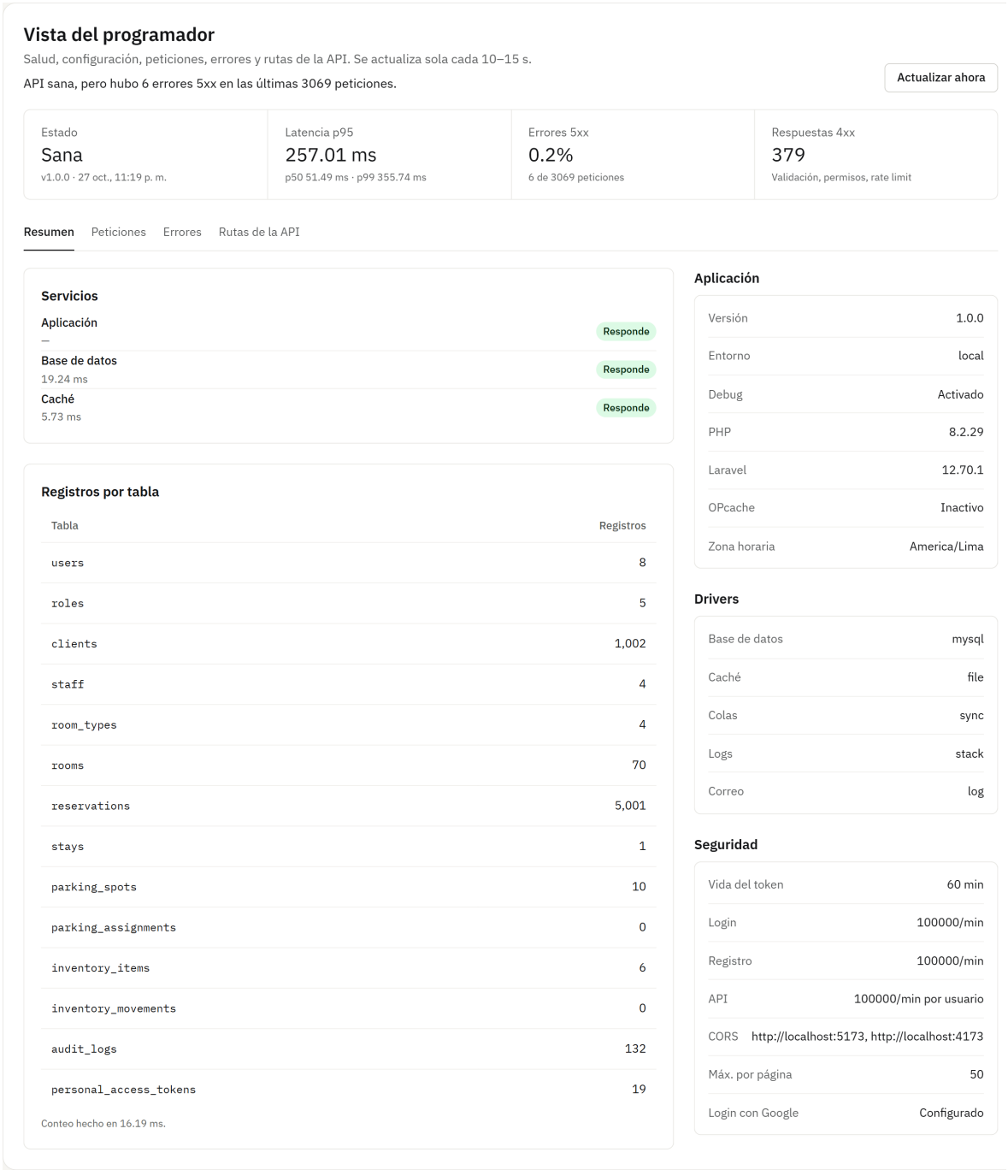
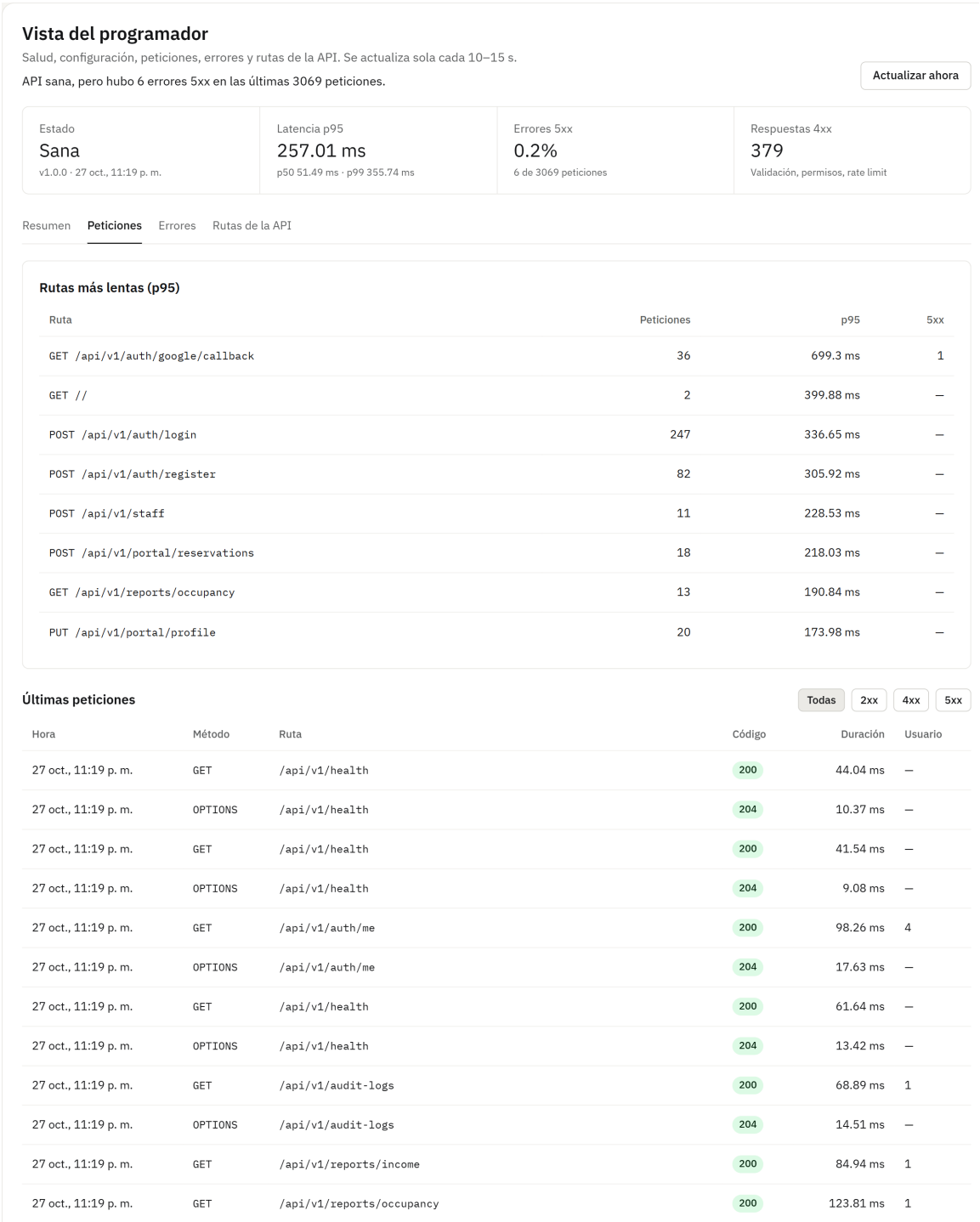


Figure 43. Developer view · Summary

Developer view · Requests

The slowest routes by p95 and the latest requests with their code and duration (the top part is shown).



Vista del programador

Salud, configuración, peticiones, errores y rutas de la API. Se actualiza sola cada 10–15 s.

API sana, pero hubo 6 errores 5xx en las últimas 3069 peticiones.

Actualizar ahora

Estado	Latencia p95	Errores 5xx	Respuestas 4xx
Sana	257.01 ms	0.2%	379
v1.0.0 · 27 oct., 11:19 p. m.	p50 51.49 ms · p99 355.74 ms	6 de 3069 peticiones	Validación, permisos, rate limit

Resumen | Peticiones | Errores | Rutas de la API

Rutas más lentas (p95)

Ruta	Peticiones	p95	5xx
GET /api/v1/auth/google/callback	36	699.3 ms	1
GET //	2	399.88 ms	—
POST /api/v1/auth/login	247	336.65 ms	—
POST /api/v1/auth/register	82	305.92 ms	—
POST /api/v1/staff	11	228.53 ms	—
POST /api/v1/portal/reservations	18	218.03 ms	—
GET /api/v1/reports/occupancy	13	190.84 ms	—
PUT /api/v1/portal/profile	20	173.98 ms	—

Últimas peticiones

Todas | 2xx | 4xx | 5xx

Hora	Método	Ruta	Código	Duración	Usuario
27 oct., 11:19 p. m.	GET	/api/v1/health	200	44.04 ms	—
27 oct., 11:19 p. m.	OPTIONS	/api/v1/health	204	10.37 ms	—
27 oct., 11:19 p. m.	GET	/api/v1/health	200	41.54 ms	—
27 oct., 11:19 p. m.	OPTIONS	/api/v1/health	204	9.08 ms	—
27 oct., 11:19 p. m.	GET	/api/v1/auth/me	200	98.26 ms	4
27 oct., 11:19 p. m.	OPTIONS	/api/v1/auth/me	204	17.63 ms	—
27 oct., 11:19 p. m.	GET	/api/v1/health	200	61.64 ms	—
27 oct., 11:19 p. m.	OPTIONS	/api/v1/health	204	13.42 ms	—
27 oct., 11:19 p. m.	GET	/api/v1/audit-logs	200	68.89 ms	1
27 oct., 11:19 p. m.	OPTIONS	/api/v1/audit-logs	204	14.51 ms	—
27 oct., 11:19 p. m.	GET	/api/v1/reports/income	200	84.94 ms	1
27 oct., 11:19 p. m.	GET	/api/v1/reports/occupancy	200	123.81 ms	1

Figure 44. Developer view · Requests

Developer view · Errors

Server errors, searchable by the incident code the user sees on the 500 screen. The real incident JJA05WR4CG from section 6 appears here.

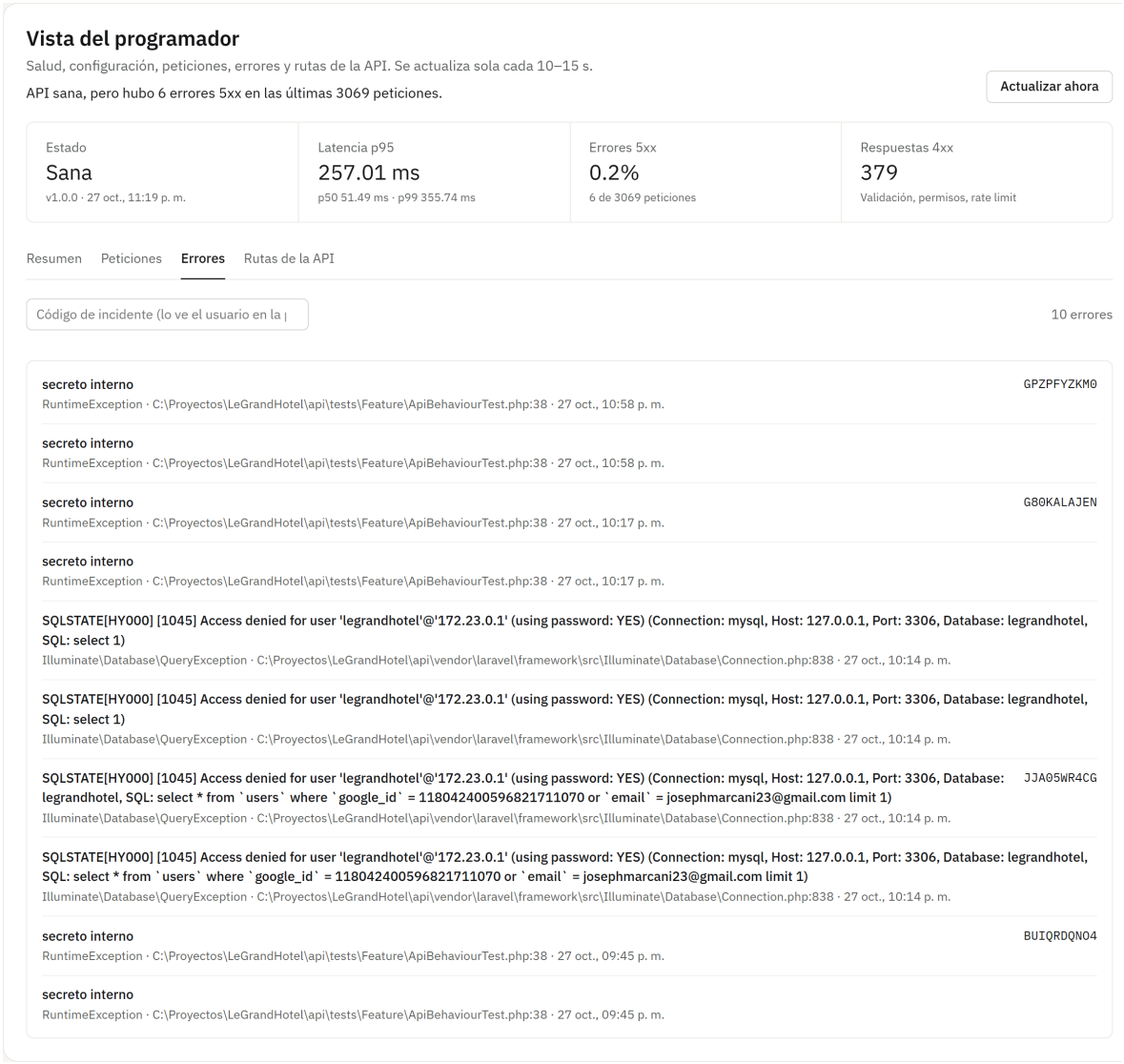
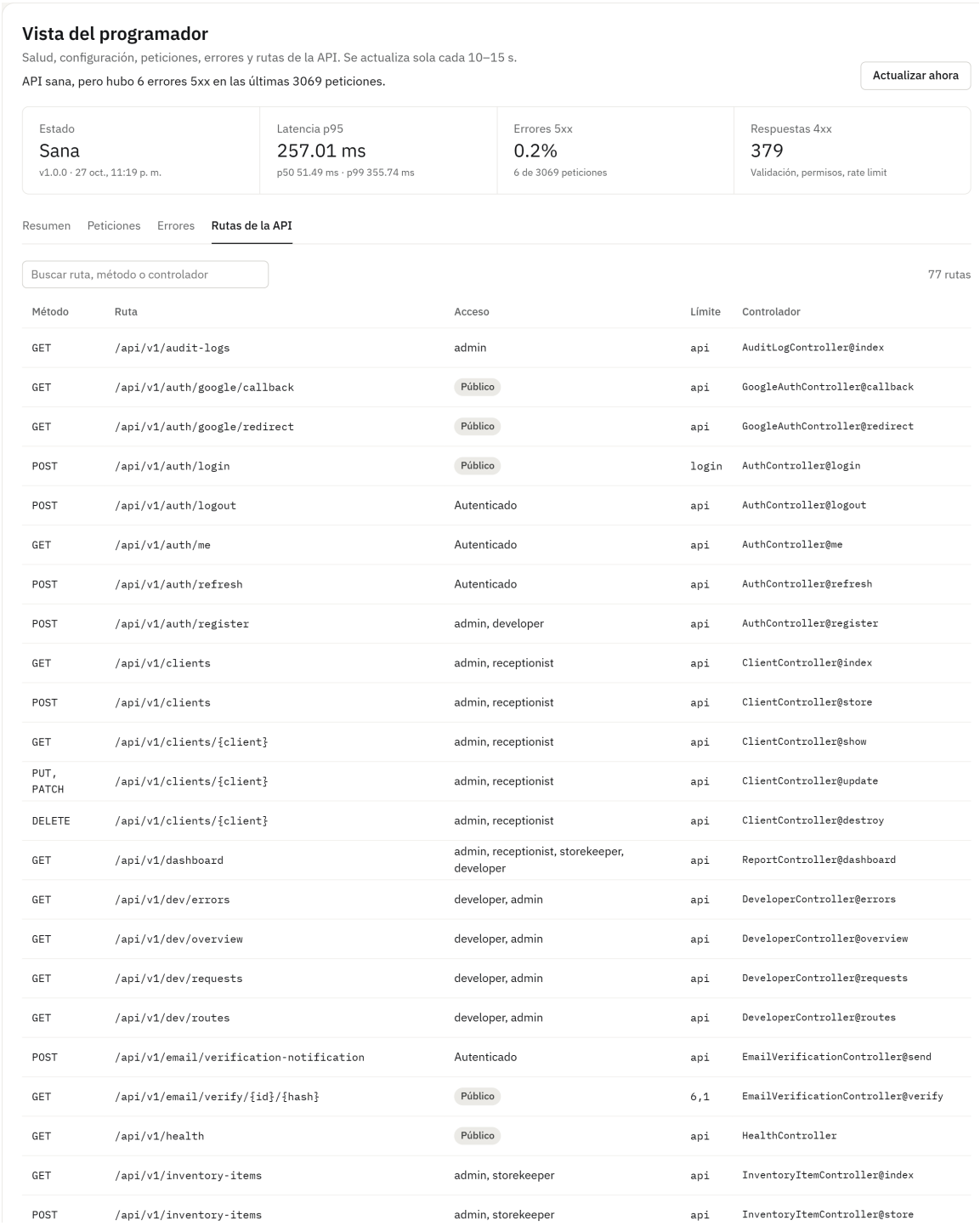


Figure 45. Developer view · Errors

Developer view · API routes

The 77 routes with their method, whether they require a session, the roles that can use them and their rate limit (the top part is shown).



Vista del programador

Salud, configuración, peticiones, errores y rutas de la API. Se actualiza sola cada 10–15 s.

API sana, pero hubo 6 errores 5xx en las últimas 3069 peticiones.

Actualizar ahora

Estado	Latencia p95	Errores 5xx	Respuestas 4xx
Sana	257.01 ms	0.2%	379
v1.0.0 · 27 oct., 11:19 p. m.	p50 51.49 ms · p99 355.74 ms	6 de 3069 peticiones	Validación, permisos, rate limit

Resumen | Peticiones | Errores | **Rutas de la API**

Buscar ruta, método o controlador — 77 rutas

Método	Ruta	Acceso	Límite	Controlador
GET	/api/v1/audit-logs	admin	api	AuditLogController@index
GET	/api/v1/auth/google/callback	Público	api	GoogleAuthController@callback
GET	/api/v1/auth/google/redirect	Público	api	GoogleAuthController@redirect
POST	/api/v1/auth/login	Público	login	AuthController@login
POST	/api/v1/auth/logout	Autenticado	api	AuthController@logout
GET	/api/v1/auth/me	Autenticado	api	AuthController@me
POST	/api/v1/auth/refresh	Autenticado	api	AuthController@refresh
POST	/api/v1/auth/register	admin, developer	api	AuthController@register
GET	/api/v1/clients	admin, receptionist	api	ClientController@index
POST	/api/v1/clients	admin, receptionist	api	ClientController@store
GET	/api/v1/clients/{client}	admin, receptionist	api	ClientController@show
PUT, PATCH	/api/v1/clients/{client}	admin, receptionist	api	ClientController@update
DELETE	/api/v1/clients/{client}	admin, receptionist	api	ClientController@destroy
GET	/api/v1/dashboard	admin, receptionist, storekeeper, developer	api	ReportController@dashboard
GET	/api/v1/dev/errors	developer, admin	api	DeveloperController@errors
GET	/api/v1/dev/overview	developer, admin	api	DeveloperController@overview
GET	/api/v1/dev/requests	developer, admin	api	DeveloperController@requests
GET	/api/v1/dev/routes	developer, admin	api	DeveloperController@routes
POST	/api/v1/email/verification-notification	Autenticado	api	EmailVerificationController@send
GET	/api/v1/email/verify/{id}/{hash}	Público	6,1	EmailVerificationController@verify
GET	/api/v1/health	Público	api	HealthController
GET	/api/v1/inventory-items	admin, storekeeper	api	InventoryItemController@index
POST	/api/v1/inventory-items	admin, storekeeper	api	InventoryItemController@store

Figure 46. Developer view · API routes

4.5 Guest

The guest with their own account: signs in with Google, completes their DNI and phone once and books online. They see nothing of the staff panel.

Complete registration (before the menu)

The first time they sign in with Google this screen appears, still without a menu. Name and email come from Google; the guest enters their DNI (8 digits) and phone (9 digits). If their DNI was already registered at the front desk, that record is linked.

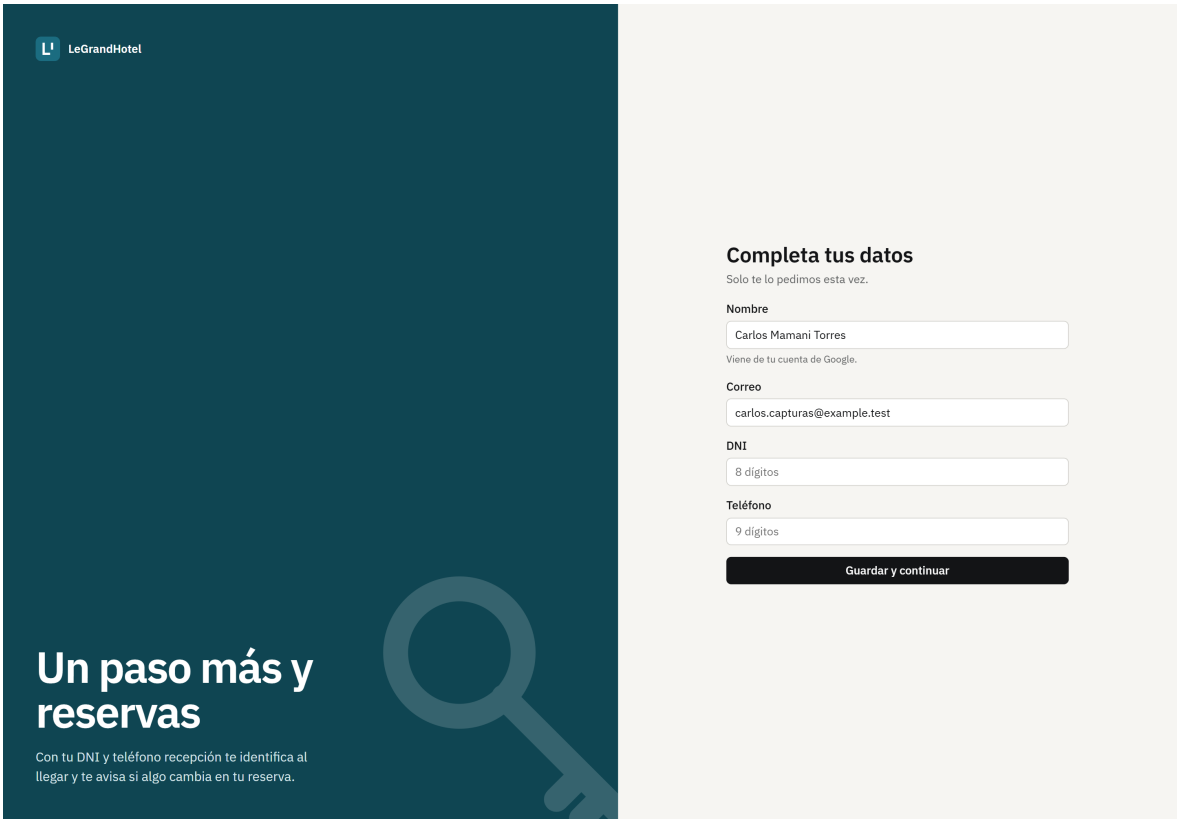


Figure 47. Complete registration (before the menu)

Guest menu

Their “Mi cuenta” menu has two options: Book («Reservar») and My bookings («Mis reservas»).

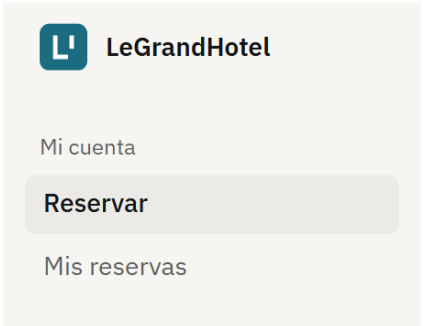


Figure 48. Guest menu.

Book («Reservar»)

They choose arrival, departure and number of guests; they see each room type with its amenities, how many are free and the total of their stay. Types that do not fit that many guests do not appear.

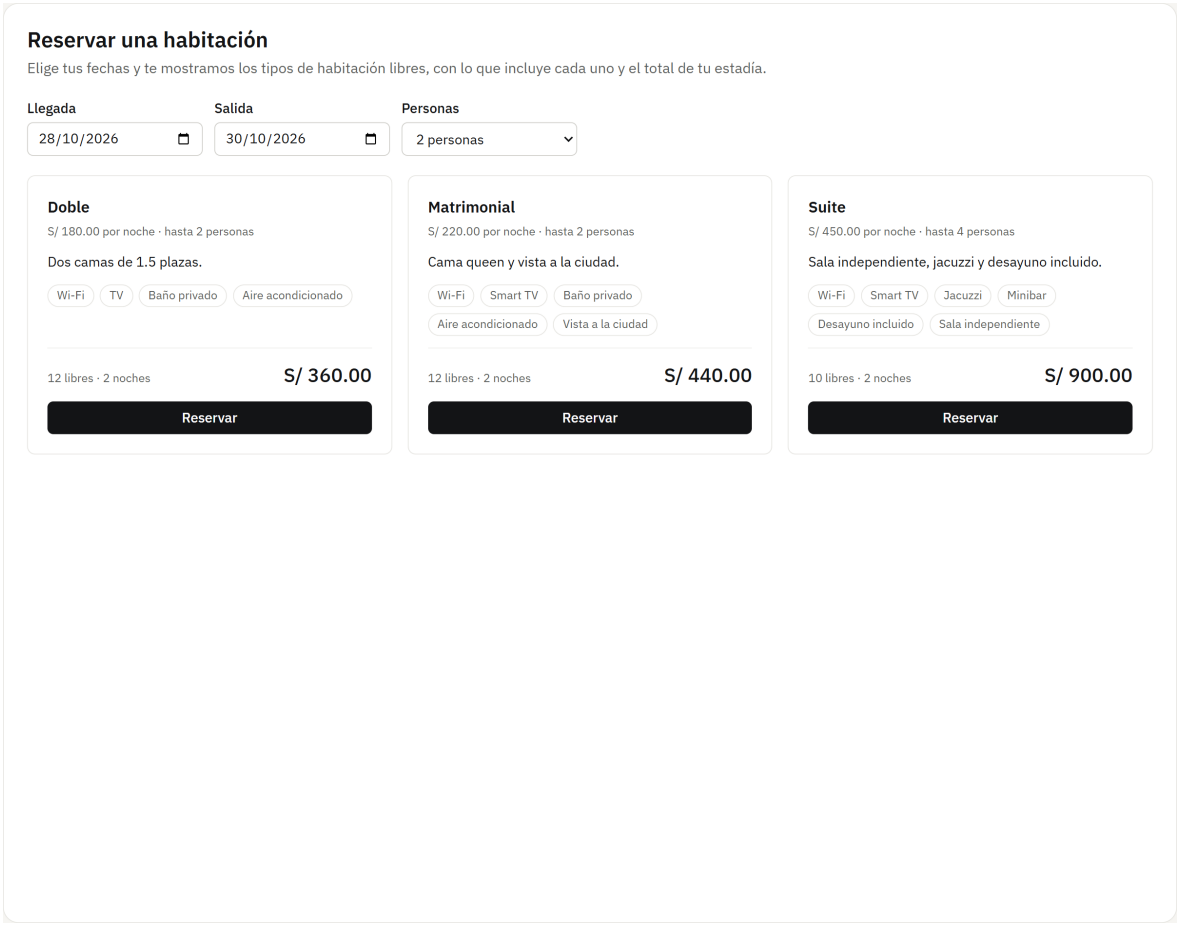


Figure 49. Book («Reservar»)

Pop-up window: Confirm reservation

Summary with the type, dates, nights and total before confirming. On confirmation, the hotel assigns a free room of that type.



Figure 50. Pop-up window: Confirm reservation

My bookings («Mis reservas»)

Their reservations with room, dates, nights, total and status. They can cancel a confirmed reservation as long as at least one day remains before arrival.

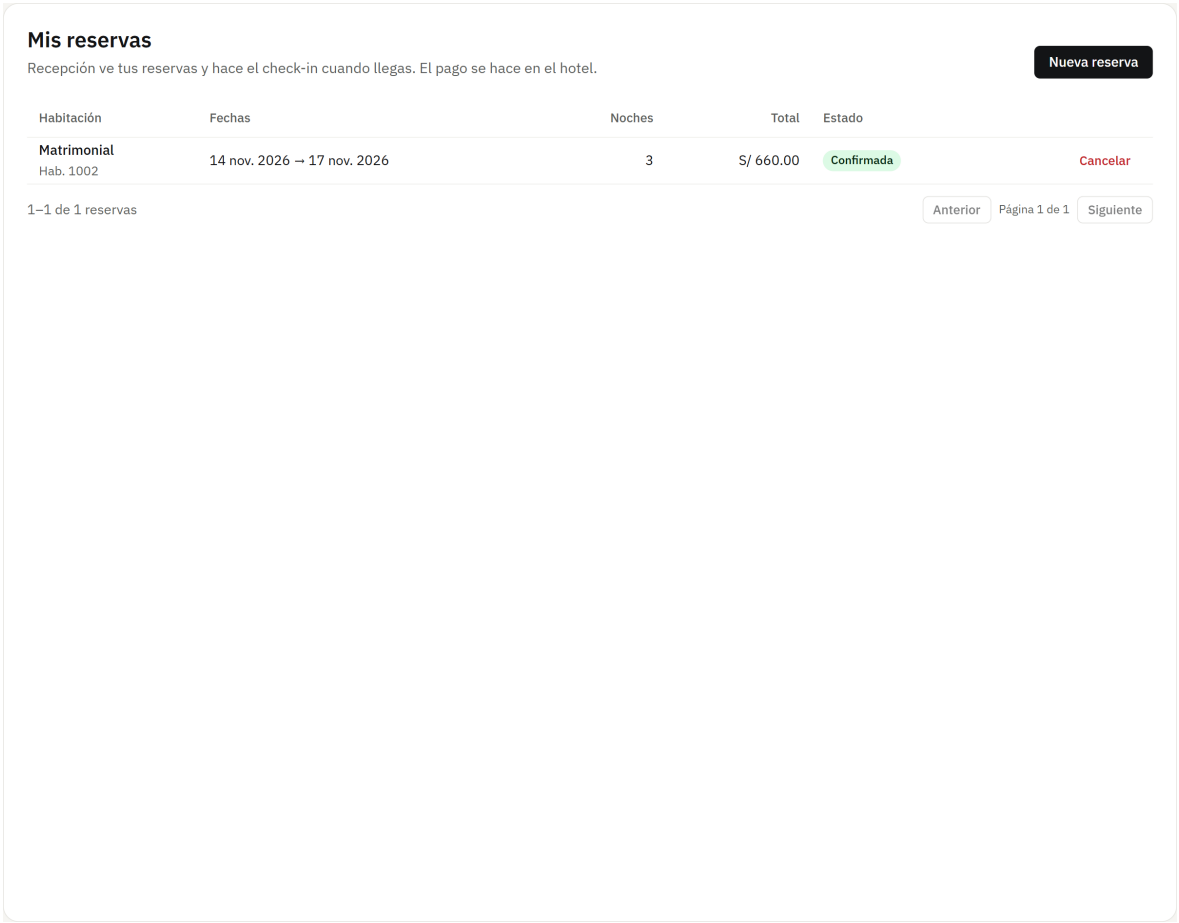


Figure 51. My bookings («Mis reservas»)

5. Error screens

Every API error response has its own screen, with the same style as the page, a clear message and a way out (go home, reload or sign in). A single interceptor in web/src/lib/api.ts decides which one to show, so they behave the same in every view.

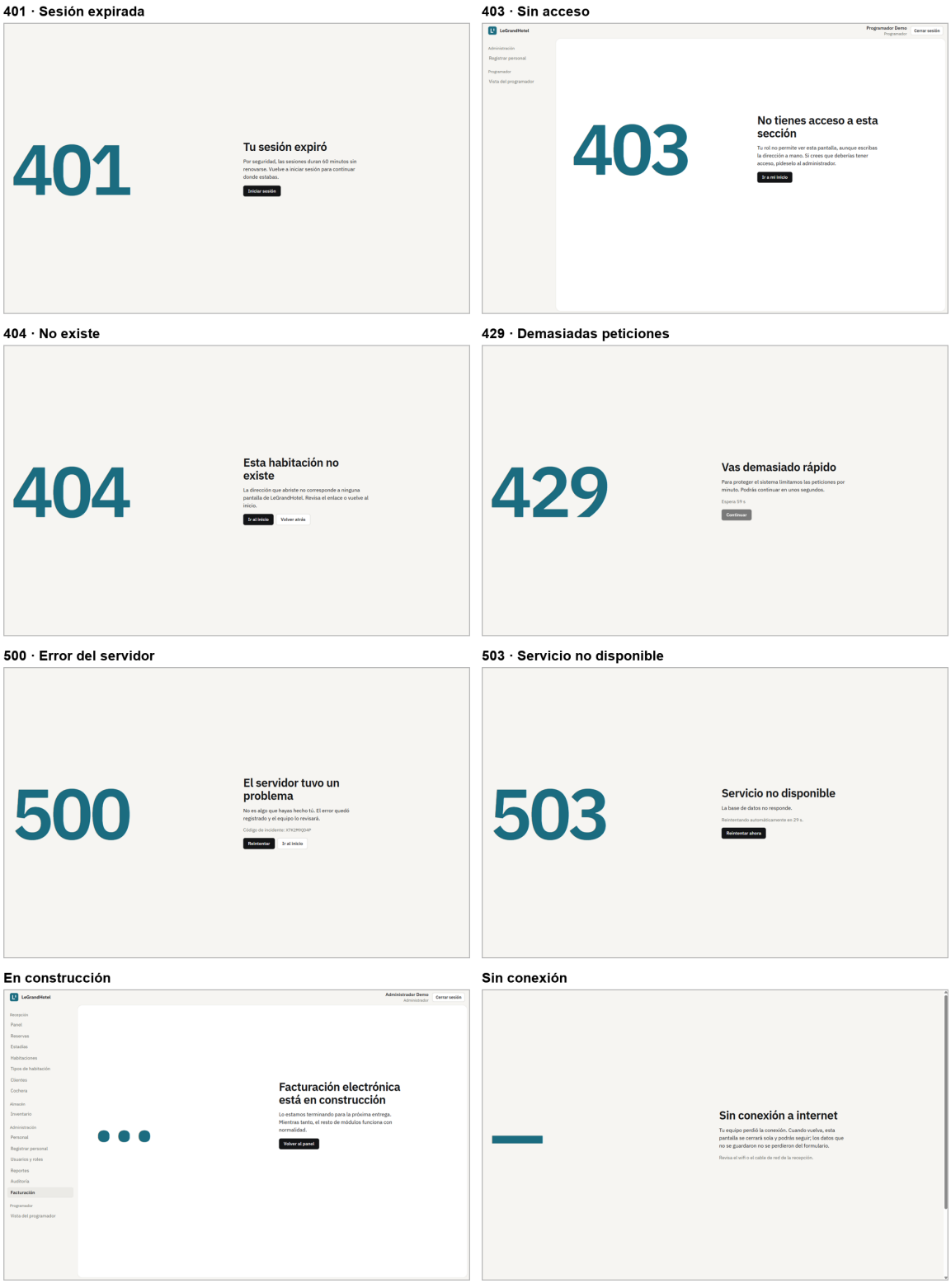


Figure 52. 401, 403, 404, 429, 500, 503, under construction and offline screens (Spanish interface).

Screen	When it appears
401 · Session expired	The token expired (60 minutes) or the session was closed elsewhere
403 · No access	The role is not allowed to open that view or do that action
404 · Not found	The address does not exist or the record was deleted

Screen	When it appears
429 · Too many requests	The limit was exceeded (for example 5 logins per minute); it shows how long to wait
500 · Server error	Something failed; it shows the incident code to look it up in the Developer view
503 · Unavailable	The API does not respond; it retries by itself every 30 seconds
Under construction	A planned module that is not built yet (Billing)
Offline	The computer lost internet; it comes back by itself when reconnected

6. Incident log

Every outage or failure, even in the local environment or in tests, is recorded the same day in docs/incidentes.md with its cause and solution. The log now has 8 incidents; 6 are from October 27, while repeating the load tests, setting up Google sign-in and reviewing the Developer view.

Date	What failed	Cause	Solution	Time
2026-10-26	Login returned 500 INTERNAL_ERROR with incident code XG1R7UASWT	php artisan serve starts child PHP processes without the pdo_sqlite extension, which was loaded with -d only in the parent ("could not find driver")	The local server is started with php -d extension=pdo_sqlite -S … (.claude/launch.json). The error came back as JSON without traces, as the plan requires	~2 min
2026-10-26	The frontend showed the 503 screen "Could not reach the server"	During the previous change the PHP server pointed to a server.php that looked for index.php in the wrong folder	api/public/index.php is used as the server router. The 503 screen retried by itself every 30 s	~1 min
2026-10-27	First k6 series: 100% failed requests	The PHP server listened only on IPv6 (::1) and k6 resolved localhost to IPv4 (127.0.0.1)	BASE_URL=http://[::1]:8000/api/v1; the invalid results were deleted and the 5 runs repeated	~5 min
2026-10-27	The first Google sign-in returned 500 INTERNAL_ERROR (incident JJA05WR4CG) and /health reported the database as down	While editing api/.env to add GOOGLE_CLIENT_ID and GOOGLE_CLIENT_SECRET, one character of DB_PASSWORD was deleted: MySQL answered "Access denied for user legrandhotel"	DB_PASSWORD was restored to match MYSQL_PASSWORD in docker-compose.yml; /health went back to ok. The incident code located the cause in storage/logs in seconds	~1 min
2026-10-27	After signing in with Google the browser showed ERR_INVALID_REDIRECT	FRONTEND_URL lists two comma-separated domains (for CORS) and the Google callback and email verification used the whole value as the return address	New App\Support\Frontend::url() redirects to the first domain; covered by a test in ClientPortalTest	~3 min
2026-10-27	k6 stopped in setup() with "The API is not healthy before the test: 503"	The nginx container from docker compose was using port 8000, so k6 talked to that Nginx (with no PHP behind it) instead of the API	docker compose down and start only MySQL (docker compose up -d mysql); the k6 tutorial explains it in step 2	~10 min
2026-10-27	After coming back from Google the page stayed blank with #token=… in the address	The application did not start in that Brave tab (the API never received /auth/me), probably because of its Shields; the same flow worked in Chrome	Use Chrome or turn off Shields for localhost; the whole flow was verified with a test account	~15 min

Date	What failed	Cause	Solution	Time
2026-10-27	The "Errors" tab of the Developer view showed the 500 screen with code WEB-MVRL1WBX	Laravel stores some exceptions in the log as an object (class, message, file) and the screen tried to show them as text (React error)	RequestLogReader now always returns the class as text; new test in DeveloperViewTest	~5 min

How the incident code helped: when the API fails, it answers 500 with a code (for example JJA05WR4CG) that the user sees on screen. The same code is stored in the API's JSON log, and the Developer view finds it in seconds (figure in section 4.4). That is how we found that the problem with the first Google sign-in was the database password, not Google.

API log (storage/logs/laravel.json), summarized

```
{"message": "SQLSTATE[HY000] [1045] Access denied for user 'legrandhotel'…",
 "context": {"incident_id": "JJA05WR4CG", "exception":
"Illuminate\\Database\\QueryException"},
 "level_name": "ERROR", "datetime": "2026-10-27T22:14:14-05:00"}
```

7. Performance and stress tests with k6

k6 simulates many "virtual users" (VU) using the API at the same time and measures how long each response takes. Each script has goals (thresholds): if they are not met, k6 marks the test with ✗ and exits with code 99. The runs were made on October 27, 2026 on the team's laptop, with the API on PHP's built-in server (php -S, which handles one request at a time) and a database with 5,000 reservations.

Test	What it does	Why
1. Smoke	5 VU for 30 s request /health and a protected route without a token	Confirm the API, database and cache work before adding load
2. Login limit	6 login attempts with a wrong password	Show the brute-force protection (429)
3 to 6. Steps	Front desk work (70% queries, 20% login, 10% new reservation) with 5, 10, 25 and 50 VU, 1 minute each	See when the API slows down and whether errors appear as load grows

7.1 Smoke test

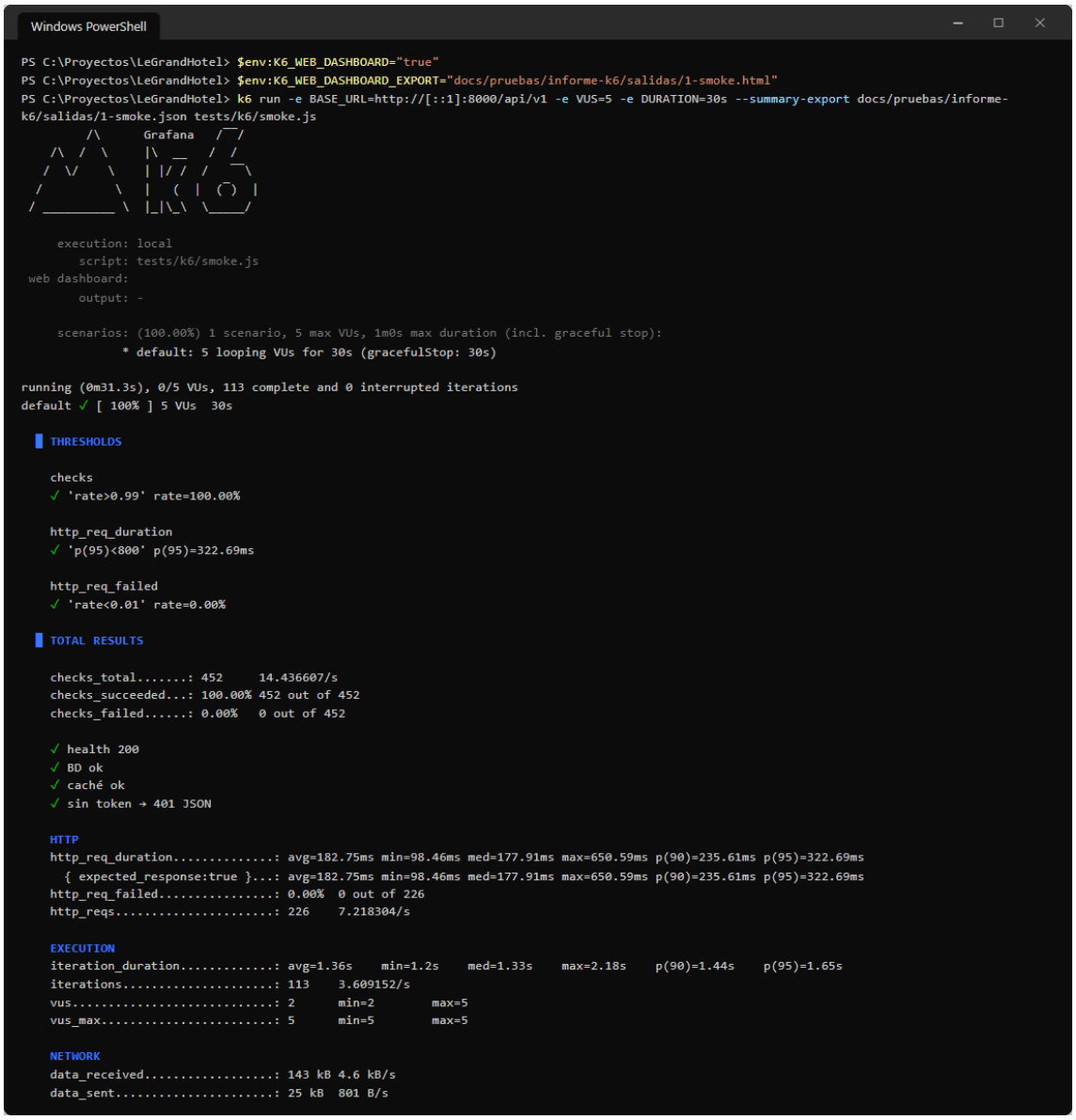


Figure 53. Smoke test command and k6 output.

Interpretation: all three goals pass with ✓. 95% of the responses took less than 323 ms (goal: 800 ms), there were no failed requests (0.00%) and all 452 checks passed: the API, database and cache respond, and a protected route without a token returns 401 in JSON.

7.2 Login attempt limit

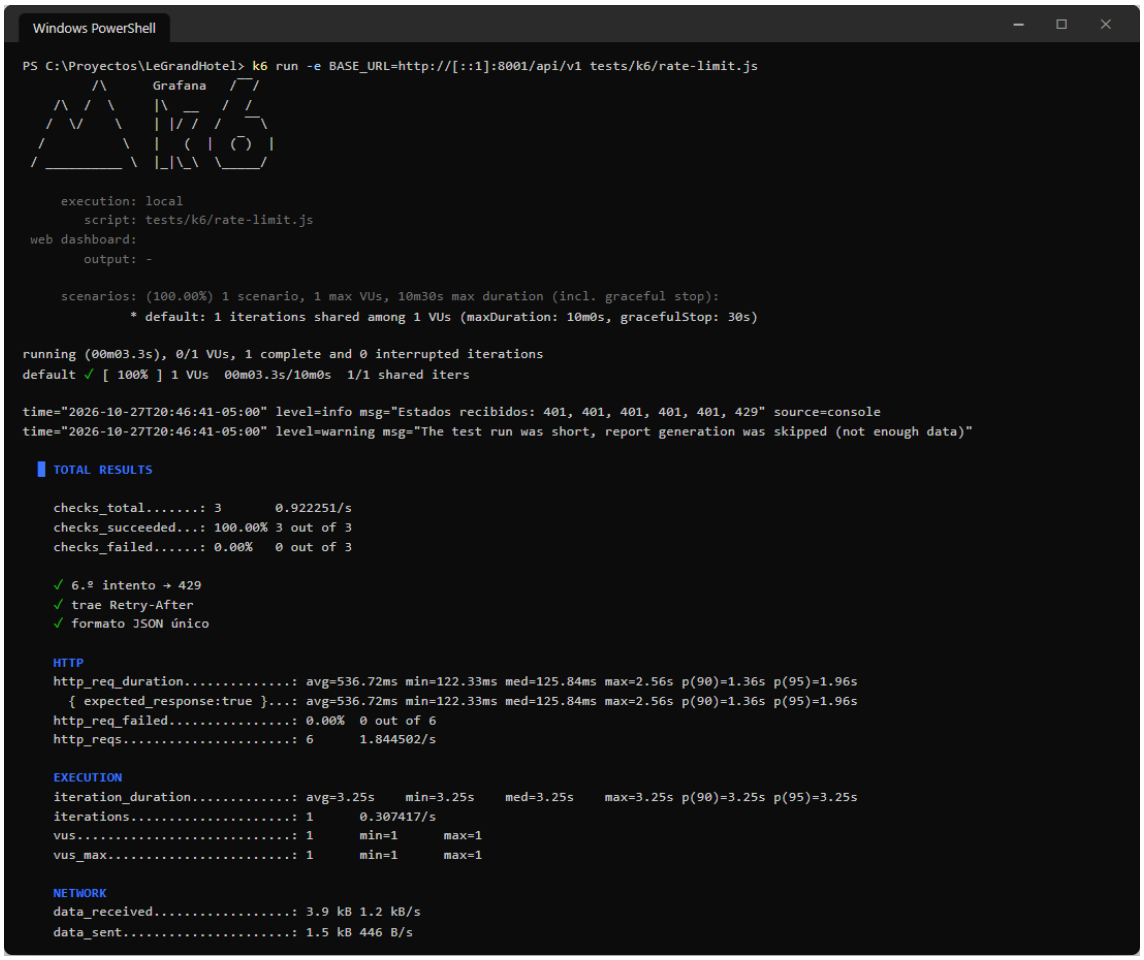


Figure 54. Login limit test: the sixth attempt gets 429.

Interpretation: “Estados recibidos: 401, 401, 401, 401, 401, 429”. The first five attempts answer 401 (wrong password) and the sixth 429, with the Retry-After header and the TOO_MANY_REQUESTS code. Nobody can try passwords without limit.

7.3 Load steps (5, 10, 25 and 50 users)



Figure 55. 5-user step: command and output.

Interpretation: with 5 users every request succeeds (0% errors and all checks ✓), but the speed goal fails with ✗: the p95 was 610 ms and the goal is 500 ms. The final line “thresholds … have been crossed” is k6 reporting that the test did not pass.



Figure 56. 50-user step: command and output.

Interpretation: with 50 users there are still no errors (0.00%), but the p95 rises to 9,346 ms. The API does not go down: it answers everything, but requests wait their turn.

Users	Requests	Req/s	Median	p95	Maximum	Errors	Goal p95 < 500 ms
5	308	4.8	304 ms	610 ms	1,131 ms	0.00%	✗
10	320	4.9	1,155 ms	1,798 ms	2,037 ms	0.00%	✗
25	367	4.8	3,936 ms	4,629 ms	5,010 ms	0.00%	✗
50	452	4.8	7,922 ms	9,346 ms	9,823 ms	0.00%	✗

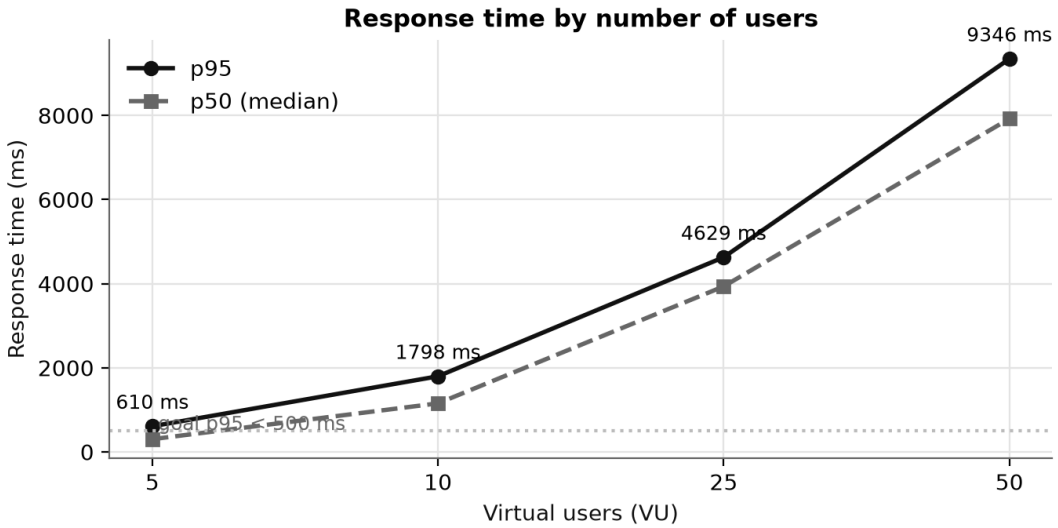


Figure 57. Response time (median and p95) by number of users.

Chart interpretation: the horizontal axis is the number of simultaneous users and the vertical axis the response time. The dotted line is the 500 ms goal. Both curves rise almost in a straight line: from 5 to 50 users the p95 grows from 610 ms to 9,346 ms. Growing in proportion to the users is the sign of a queue: each new user waits for the previous ones.

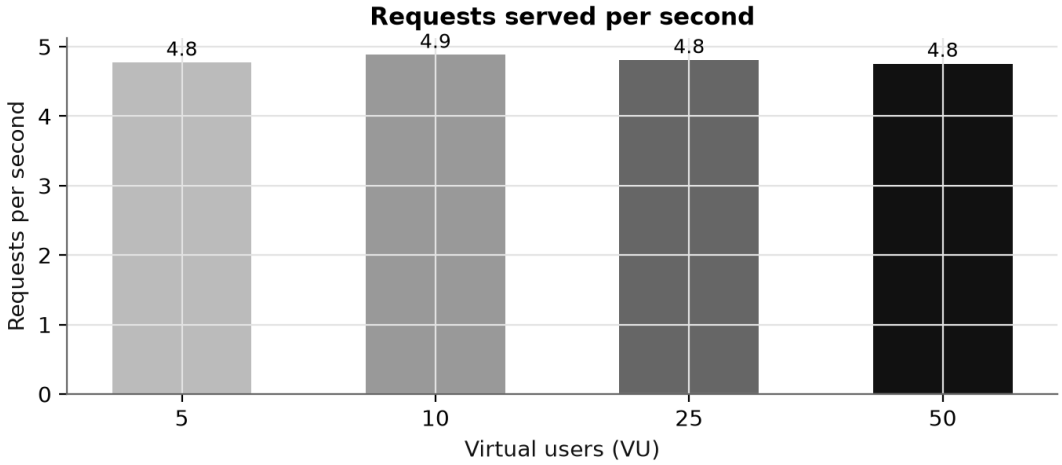


Figure 58. Requests served per second in each step.

Chart interpretation: the bars barely change (from 4.8 to 4.8 requests per second). Even with ten times more users, the API serves the same per second: it reached its ceiling. Together with the previous chart it confirms the bottleneck.

7.4 k6 HTML report (50-user step)

k6 also produces a report with charts of the minute the test lasted (salidas/3-escalon-50vu.html). Its three charts are shown separately. The horizontal axis is the time of day (20:50:20 to 20:51:40) and each point summarizes 10 seconds.

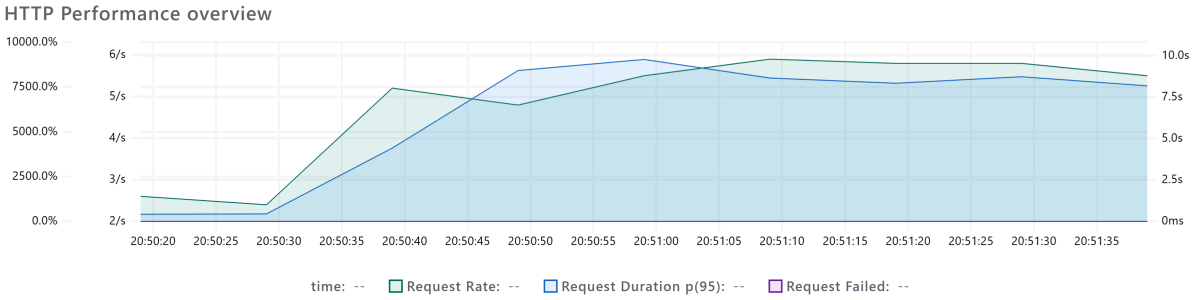


Figure 59. HTTP Performance overview: requests per second, p95 and failed requests.

What it shows: three measures at once, each with its own axis. The green line is the number of requests per second (axis from 2/s to 6/s); the blue one, the p95 response time (right axis, 0 to 10 s); the purple one, the percentage of failed requests (left axis).

Interpretation: until 20:50:30 there is almost no load and the p95 is low. From 20:50:35, when the users come in, the p95 jumps to about 9.7 s at 20:51:00 and stays between 8 and 9 s for the rest of the test (overall p95: 9,346 ms). Requests per second rise only to about 5.5/s and flatten: the API serves no more even though more users are waiting. The purple line stays at 0% the whole time: none of the 452 requests failed. Conclusion: the API slows down, but it does not go down.

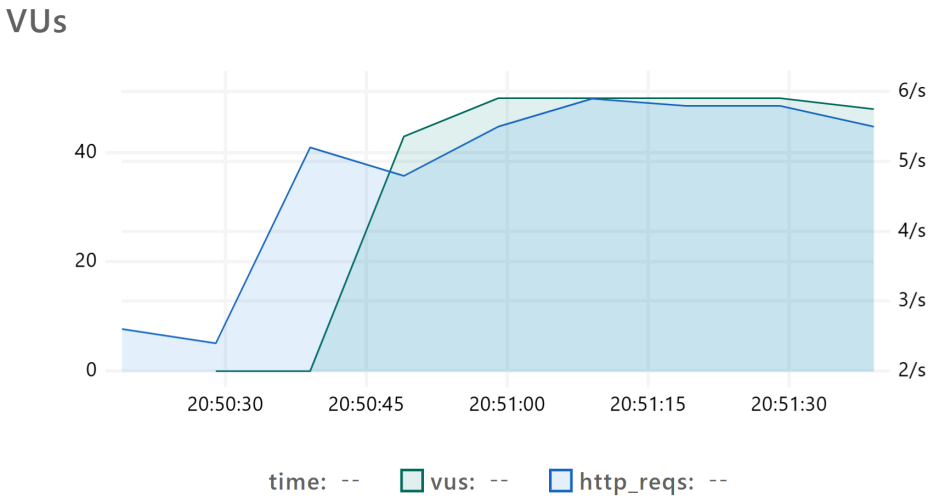


Figure 60. VUs: active virtual users and requests per second.

What it shows: the green line is the active virtual users (left axis, 0 to 50); the blue one, requests per second (right axis).

Interpretation: the three stages of the step are visible: the ramp from 0 to 50 users between 20:50:38 and 20:51:00, the plateau with 50 users until 20:51:30 and the start of the ramp-down at the end. Before the ramp there are already requests with 0 users: that is the setup, which signs in the 50 test receptionists. The key point is that the blue line reaches its ceiling (about 5.5/s) with around 40 users and does not rise further at 50: the extra users only add waiting. It is the same bottleneck signal as the previous chart.

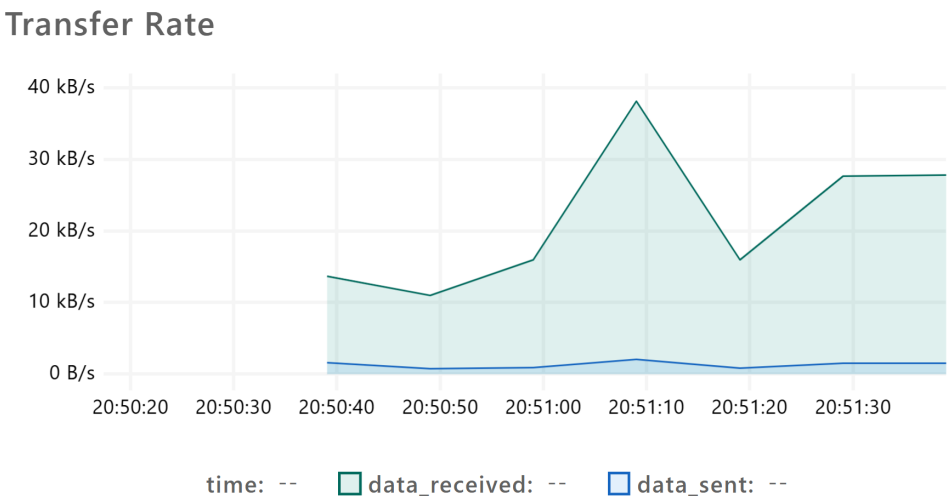


Figure 61. Transfer Rate: data received and sent per second.

What it shows: how much data k6 received from the API (data_received, green) and how much it sent (data_sent, blue), in kilobytes per second.

Interpretation: data received varies between 11 and 38 kB/s, with a peak at 20:51:10. It varies because it depends on what each user requested: a list of reservations or free rooms weighs more than a login. Data sent stays around 1 to 2 kB/s because requests are small (a token and a few fields). In total, 1.8 MB were received and 108 kB sent over the whole minute: the network is far from being the limit, so the slowness comes from the API processing one request at a time, not from the amount of data.

7.5 Test conclusions

- Correct and stable: across the 6 runs there was not a single failed request or 500 error.
- Secure under load: 401 without a session and 429 when the login limit is exceeded, always in the API's JSON format.
- Slow on the laptop because of PHP's development server, which handles one request at a time. The fix is to run the API with several PHP processes in parallel and code caching (OPcache) on the final server, and repeat these same tests there, with more users, once the server is defined.

8. Android and iOS app design

LeGrandHotel will have a phone app that uses the same API and the same roles as the web, without duplicating any business rule. It covers what is convenient to do from a phone: guests book and review their reservations, the front desk handles arrivals and departures from the lobby, and the storeroom records movements in the warehouse.

Role	What they will do in the app
Guest	Sign in with Google, complete DNI and phone, book by room type, view and cancel their reservations, get a reminder one day before arrival

Role	What they will do in the app
Receptionist	“Today” screen with arrivals and departures, check-in and check-out, reservations, room status and parking
Storekeeper	See the inventory with supplies below the minimum and record stock in and out
Administrator	The summarized dashboard (occupancy, income and alerts); full management stays on the web

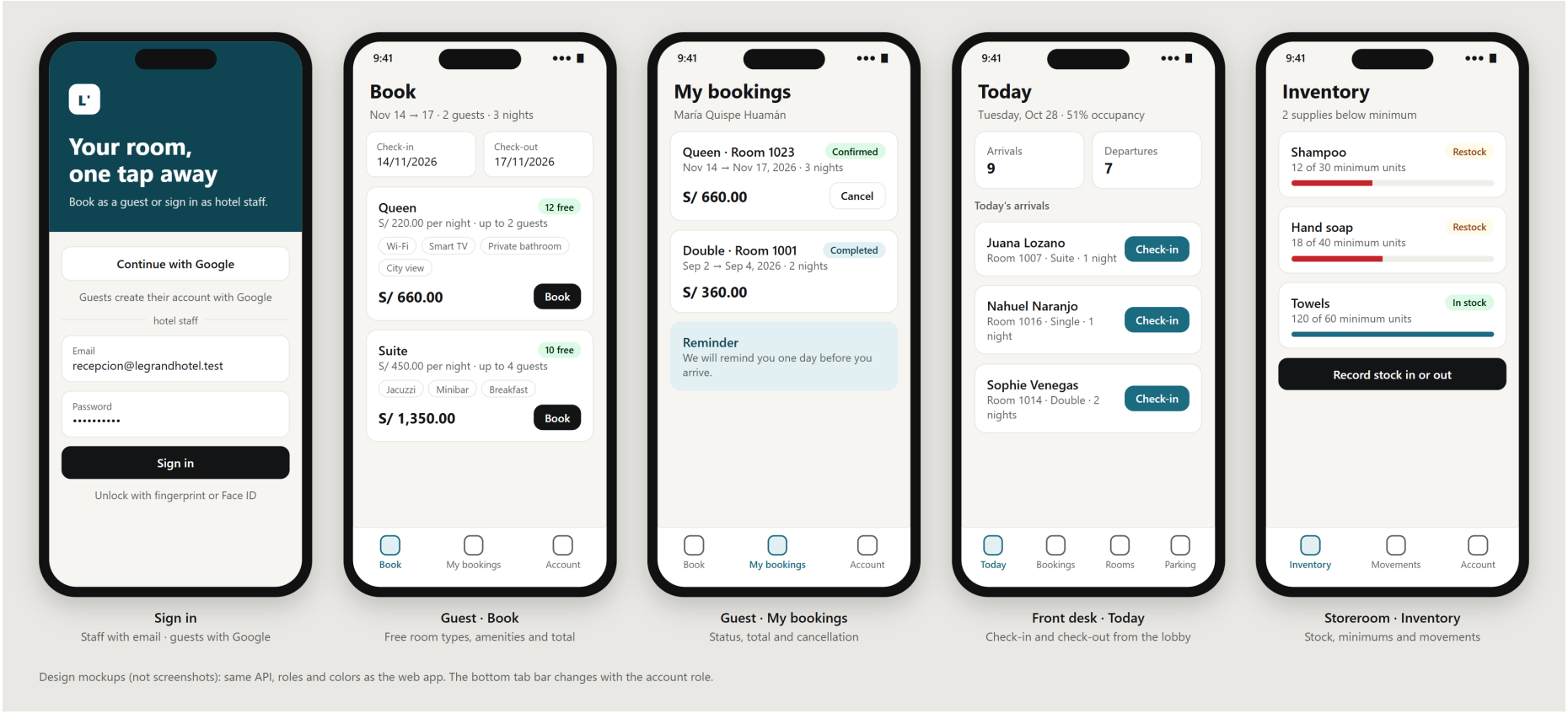


Figure 62. App mockups: sign in, the guest's Book and My bookings, the front desk's "Today" and the storeroom's Inventory.

8.1 Technology

React Native with Expo and TypeScript. A single codebase produces both apps and reuses what the team already uses on the web: React, TypeScript, TanStack Query, Axios and the same data types.

Need	In the app	Web equivalent
Navigation	Expo Router, with a tab bar based on the role	React Router
API calls	TanStack Query and Axios with the same 401/403/429/5xx handling	Same
Session storage	expo-secure-store: Keychain (iOS) and Keystore (Android)	sessionStorage
Google sign-in	expo-auth-session with the link legrandhotel://auth/callback	Socialite with redirect
Notifications	expo-notifications (arrival reminder, room ready, low stock)	—
Unlock	Fingerprint or Face ID (expo-local-authentication)	—
Tests	Jest and Testing Library; Maestro for end-to-end flows	Vitest and Playwright

Option evaluated	Pros	Cons	Decision
React Native + Expo	One codebase for Android and iOS; same language as the web; builds iOS without a Mac (EAS Build)	Very specific native features need custom modules	Chosen
Flutter	Good performance and consistent design	Another language (Dart): nothing from the web is reused	Discarded
Native (Kotlin + Swift)	Full access to the system	Two different apps: twice the work	Discarded

8.2 Security and API changes

- The token is stored in the phone's secure storage, never in plain text, and is renewed with the same POST /auth/refresh as the web.
- The app contains no secrets: the Google and database credentials stay only in the API.
- The API barely changes: login already accepts a device name (device_name) so each phone gets its own token. Still pending: registering each phone's notification token (POST /devices) and letting Google sign-in return to the app.

8.3 Build plan

Stage	What is built
1	Expo project, staff and guest login, secure session and a tab bar per role
2	Guest: complete registration, book and my bookings
3	Front desk: "Today" with check-in and check-out, reservations and rooms; Storeroom: inventory and movements
4	Notifications, fingerprint unlock, tests and a test build for Android and iOS

9. Conclusions

1. The system covers the hotel's full operation. With 77 REST endpoints and a single response format, LeGrandHotel handles reservations, stays, rooms, guests, parking, inventory, staff, reports and audit, plus online booking for guests.

2. Access is separated and protected. Staff sign in with email and password and are registered by the Administrator or the Developer; guests sign in only with Google. Each role sees only its menu, the API answers 403 even if someone types the address by hand, and there is a single Developer.

3. The server enforces the business rules. The API prevents double booking, two guests in the same room, negative stock and an occupied parking spot (409), calculates the totals and records every change in the audit log.

4. Failures are logged and fixed with evidence. The incident log and the incident code the user sees made it possible to find the cause of each failure in the API log and fix it the same day, with a new automated test when appropriate.

5. The API is stable under load and the next step is clear. In the k6 tests there was not a single error with up to 50 simultaneous users; speed on the laptop is limited by the development server and will be measured again once the server is defined. The Android and iOS app will reuse this same API.